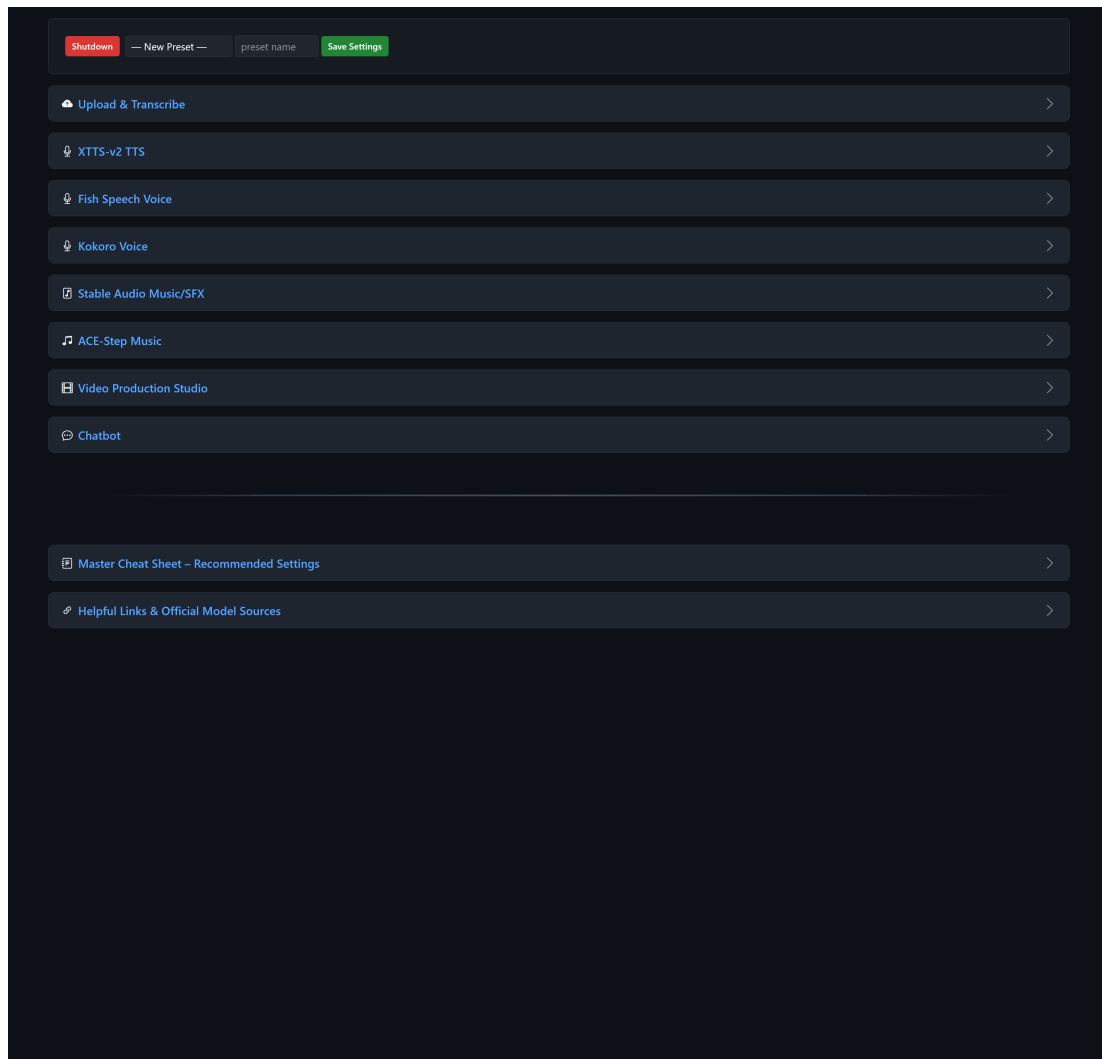


LocalSoundsAPI

User Manual

The ultimate portable, offline all-in-one audio studio

Text-to-Speech | Music Generation | Transcription | Video | Chatbot



Chapter 1: Introduction

What Is LocalSoundsAPI?

LocalSoundsAPI is a self-hosted audio production platform that brings together several powerful AI models under one roof. Instead of juggling separate tools, signing up for cloud services, or writing scripts to glue things together, you get a single web interface that handles:

- Text-to-Speech (TTS) -- Three different engines (XTTS-v2, Fish Speech, Kokoro) each with their own strengths
- Voice Cloning -- Upload a short audio sample of any voice and generate new speech in that voice
- Music Generation -- Create original music tracks from text descriptions
- Sound Effects -- Generate impacts, ambient textures, foley, and more from text prompts
- Transcription -- Convert speech to text with word-level timestamps
- Video Production -- Combine audio with images or video, add auto-generated subtitles
- AI Chatbot -- Chat with local or cloud-based language models

Everything runs on your local machine. Your audio, your voices, and your conversations stay private. The only exception is if you choose to use the optional OpenRouter cloud backend for the chatbot.

Who Is This For?

- Content creators who need voiceovers, narration, or background music
- Game developers looking for sound effects and character voices
- Podcasters and video makers who want automated transcription and subtitled videos
- Hobbyists and tinkerers who want to explore AI audio generation
- Developers who need a local TTS/audio API for their projects

How It Works

LocalSoundsAPI is a Flask web application written in Python. When you run it, a local web server starts on your machine. You open your browser, navigate to the address (usually `http://127.0.0.1:5006`), and interact with everything through the web interface.

Behind the scenes, AI models are loaded onto your GPU (or CPU) on demand. You only load what you need, and you can unload models when you're done to free up memory. This "lazy loading" approach means the app starts quickly and you control your GPU memory usage.

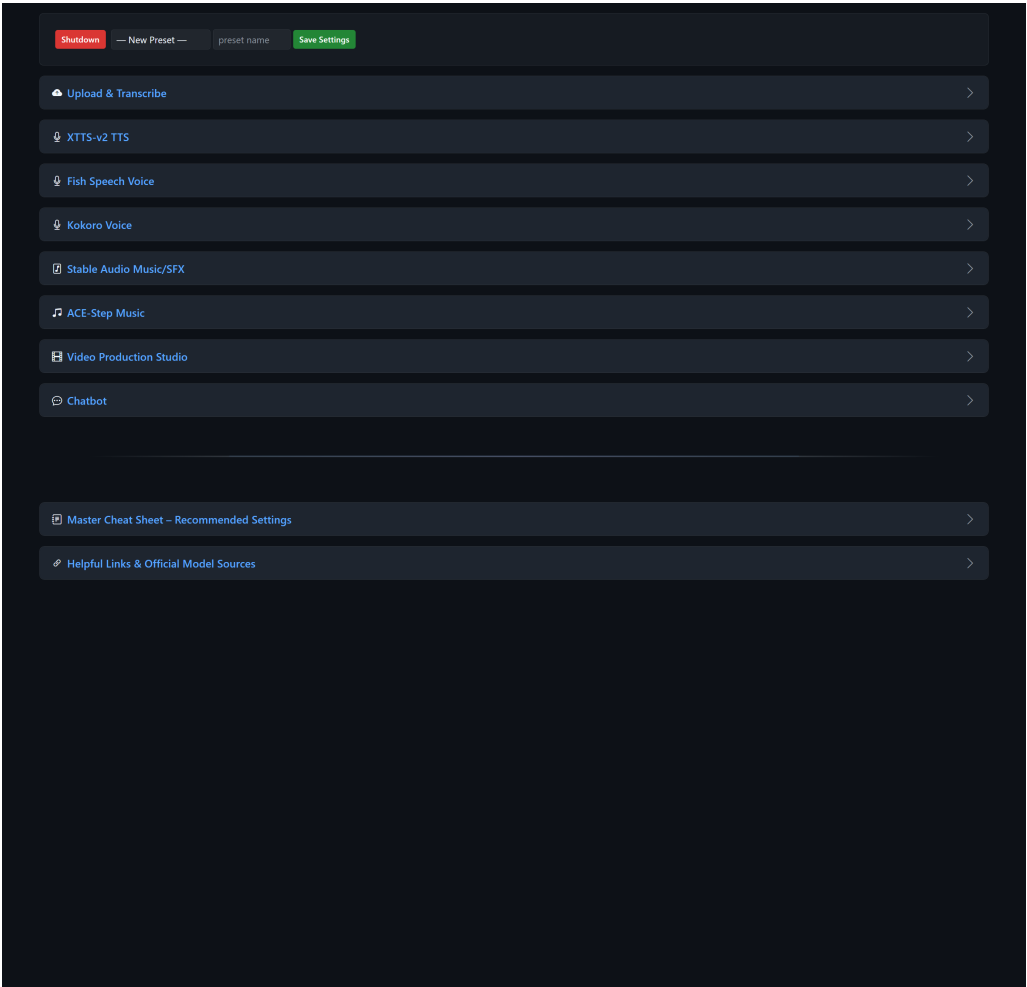
Architecture at a Glance

```
Your Browser
|
v
Flask Web Server (main.py, port 5006)
|
+-- Routes (API endpoints for each feature)
|   +-- TTS: XTTS, Fish Speech, Kokoro
|   +-- Audio: Stable Audio, ACE-Step
|   +-- Production: Upload, Transcribe, Video
|   +-- Chatbot: Local LLM, LM Studio, OpenRouter
|
+-- Models (loaded/unloaded on demand)
```

```
|   +-- XTTS-v2, Fish Speech, Kokoro, Whisper
|   +-- Stable Audio, ACE-Step, CLAP
|   +-- Llama.cpp (local LLM)
|
+-- Storage
    +-- voices/           (your reference voice samples)
    +-- output_tts/       (temporary working files)
    +-- projects_output/  (saved project files)
    +-- brain/            (chatbot memory and prompts)
    +-- settings/         (saved parameter presets)
```

Key Design Principles

1. Local-first privacy -- All processing happens on your hardware. No data leaves your machine unless you explicitly use the OpenRouter cloud backend.
 2. Load only what you need -- Models consume significant GPU memory. LocalSoundsAPI lets you load and unload each model independently, so you're never forced to have everything in memory at once.
 3. Multi-GPU support -- If you have multiple GPUs, you can assign different models to different devices. Run XTTS on GPU 0 and Whisper on GPU 1, for example.
 4. Project-based workflow -- Generated audio is organized into project folders under projects_output/. Each project keeps its audio files, transcriptions, subtitles, and videos together.
 5. Recovery built in -- Long TTS jobs track their progress. If something fails mid-way, you can recover and continue from where it left off rather than starting over.
-



The main interface with all sections collapsed. Each section expands to reveal its controls.

The complete interface with every section expanded, showing the full scope of features.

Chapter 2: Installation and Setup

System Requirements

Minimum

Component	Requirement
OS	Windows 10 or 11
CPU	Intel i5 / AMD Ryzen 5 or equivalent
RAM	8 GB
Storage	100 GB free SSD space
GPU	Optional -- CPU mode works but is much slower

Recommended

Component	Recommendation
OS	Windows 10/11 (64-bit)
CPU	Intel i9 / AMD Ryzen 9
RAM	32 GB
Storage	200+ GB SSD
GPU	NVIDIA RTX 3090 or 4090 (24 GB VRAM)
CUDA	11.8 or newer

GPU Memory (VRAM) per Model

Not all models need to be loaded at once. Load what you need, unload when done.

Model	Approximate VRAM
XTTS-v2	~6 GB
Fish Speech	~8 GB
Kokoro	~2 GB
Whisper (medium.en)	~5 GB
Stable Audio	~10 GB
ACE-Step	~7 GB
CLAP (audio scoring)	~1 GB
Llama.cpp LLM	Varies by model size

Step-by-Step Installation (Fully Portable)

LocalSoundsAPI uses a portable Python environment with all dependencies pre-built and bundled together. This isn't a standard pip install setup -- the dozens of libraries (PyTorch with CUDA, audio processing, ML frameworks, etc.) have been carefully assembled to work together without conflicts. A raw pip install -r requirements.txt will not produce a working environment.

1. Download the Repository

```
git clone https://github.com/rookiemann/LocalSoundsAPI.git
cd LocalSoundsAPI
```

Or go to the main repo page, click Code -> Download ZIP, and extract it to any folder.

2. Download and Extract the Portable Python Environment

Go to Releases and download portable-python-env-v1.7z.

Extract it directly into your project folder -- it creates the python/ subfolder containing a complete Python 3.11 installation with every dependency pre-installed (PyTorch + CUDA, Flask, audio libraries, ML frameworks, and more).

```
LocalSoundsAPI/
python/          <-- extracted from portable-python-env-v1.7z
python.exe
Scripts/
Lib/
DLLs/
...
```

This portable Python is completely isolated from any system Python you may have. No PATH changes, no conflicts, no virtual environments needed.

3. Download and Extract Portable Tools

From the same Releases page, download bin.zip.

Extract it into the existing bin/ folder inside your project. This populates the three required external tools:

```
bin/
ffmpeg/bin/      -- ffmpeg.exe, ffprobe.exe
rubberband/      -- rubberband.exe (for pitch/speed adjustment)
espeak-ng/       -- libespeak-ng.dll + espeak-ng-data/ (needed by Kokoro)
```

If you already have these tools installed system-wide, you can update the paths in config.py to point to your existing installations instead.

4. AI Models (Auto-Download on First Use)

You do not need to download models manually. When you click "Load" for any model in the web interface for the first time, it will automatically download from Hugging Face. This requires an internet connection.

Model files are stored in the models/ directory:

```
models/
XTTS-v2/         -- Coqui XTTS voice cloning model
fish-speech/     -- Fish Speech TTS model
kokoro-82m/      -- Kokoro lightweight TTS model
medium.en.pt     -- Whisper transcription model
stable-audio-open-1.0/ -- Stable Audio generation model
clap-htsat-unfused/ -- CLAP audio scoring model
```

The ACE-Step model is stored in ACE-Step/models/ace_step/.

5. Configuration (config.py)

The main configuration file is config.py in the project root. Key settings you might want to change:

Whisper model size -- Choose based on your available VRAM:

```
# Fast, ~1.5 GB VRAM (for limited GPU memory)
WHISPER_PATH = APP_ROOT / "models" / "base.en.pt"

# Best quality, ~5 GB VRAM (default)
WHISPER_PATH = APP_ROOT / "models" / "medium.en.pt"

# Maximum accuracy, ~10 GB VRAM
WHISPER_PATH = APP_ROOT / "models" / "large-v3.pt"
```

LLM settings (for the chatbot):

```
LLM_DEVICE = "0" # GPU for local Llama models ("cpu" for CPU)
LLM_DIRECTORY = r"E:\LL STUDIO" # Folder containing your .gguf model files
LMSTUDIO_API_BASE = "http://127.0.0.1:1234/v1" # LM Studio API endpoint
```

OpenRouter (optional cloud LLM):

```
OPENROUTER_API_KEY = "sk-or-v1-[your-key-here]"
```

Cleanup behavior:

```
DELETE_OUTPUT_ON_STARTUP = True # Clears the temporary output_tts/ folder each restart
```

Set this to False if you want temporary files to persist between sessions.

Verifying Your Installation

Launch the application using the batch file or launcher:

```
launcher.bat
```

Or for a single instance:

```
(portable) LocalSoundsAPI-Single.bat
```

You should see output like:

```
LocalSoundsAPI -> http://127.0.0.1:5006
[Tools] FFmpeg found at bin/ffmpeg/bin
[Tools] RubberBand found at bin/rubberband
[Tools] eSpeak-ng found at bin/espeak-ng
```

Open <http://127.0.0.1:5006> in your browser. If you see the LocalSoundsAPI interface with collapsible sections, the installation is complete.

Running on a Custom Port

Use the Launcher to set the port per instance (see Chapter 17), or use the multi-instance batch file:

```
(portable) LocalSoundsAPI-Multi.bat
```

This prompts you for the number of instances and the starting port, then launches each one in a separate window.

Running Multiple Instances

Multiple instances run on different ports for true parallel processing. Each instance is independent and can load different models. The easiest way to manage multiple instances is through the Launcher (see Chapter 17), which gives you a single window to start, stop, and monitor all instances.

Using the Launcher (Recommended)

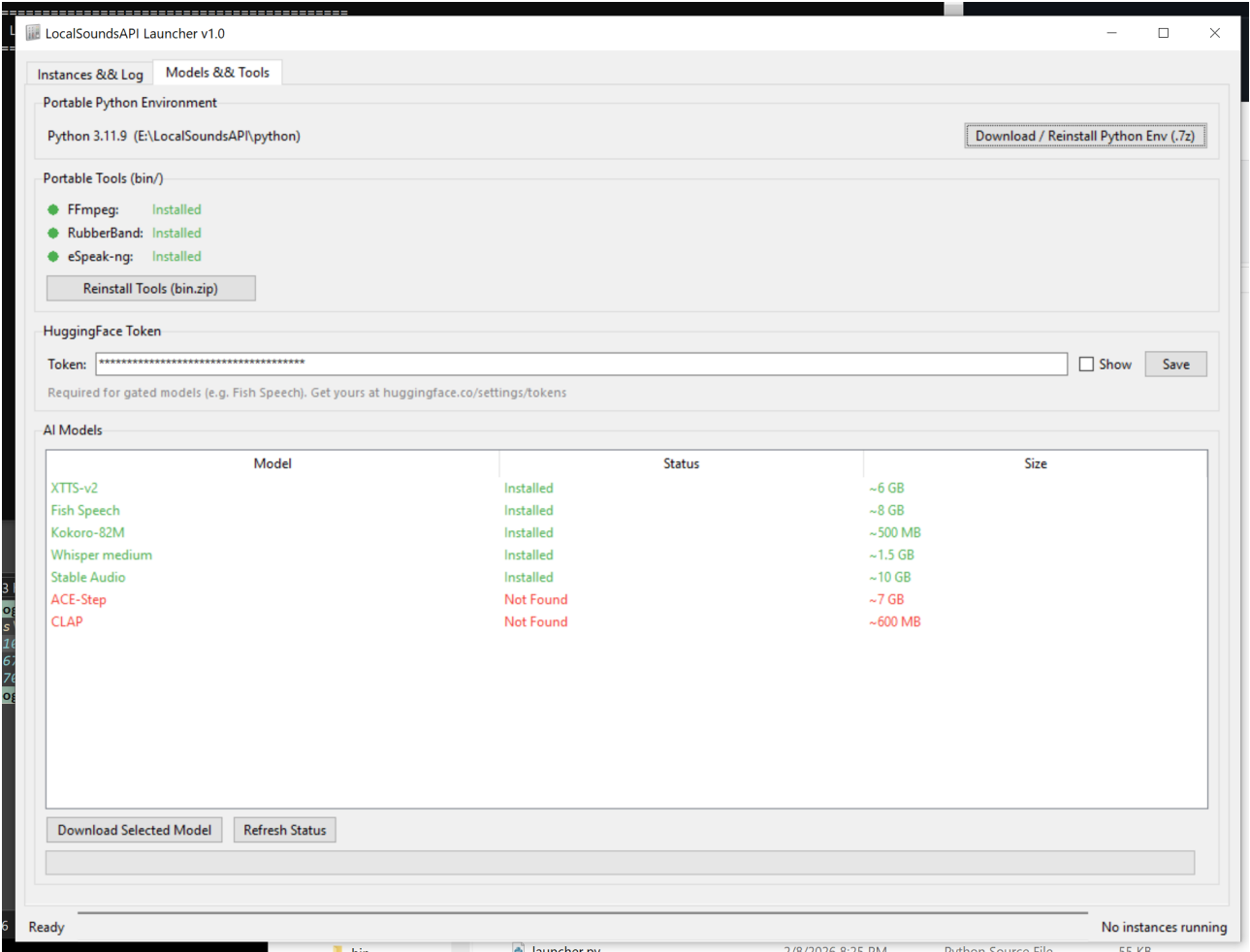
Instead of running batch files or terminal commands, you can use the Launcher -- a GUI application that manages everything from a single window:

```
launcher.bat
```

The launcher provides:

- Auto-detected GPUs -- Select which GPU each instance runs on
- Instance management -- Start, stop, and monitor multiple instances from one window
- Embedded logs -- All server output captured in a filterable log viewer (no separate cmd windows)
- Model downloads -- See which AI models are installed and download missing ones before starting
- Portable tools -- Check and install FFmpeg, RubberBand, and eSpeak-ng with one click

See Chapter 17: Launcher for the full guide.



The Launcher's Models & Tools tab showing Python environment status, portable tools (all installed), HuggingFace token entry, and AI model download status.

Chapter 3: Getting Started

Launching the Application

Open a terminal in the LocalSoundsAPI directory and run:

```
python main.py
```

Then open your browser and go to:

```
http://127.0.0.1:5006
```

The Main Interface

The entire application lives on a single page. Each feature is organized into a collapsible section that you can expand or collapse by clicking its header bar. From top to bottom, the sections are:

1. Toolbar -- Shutdown button and settings preset management
2. Upload & Transcribe -- Upload voice reference files and transcribe audio
3. XTTS-v2 TTS -- Voice cloning text-to-speech
4. Fish Speech Voice -- High-quality text-to-speech
5. Kokoro Voice -- Fast, lightweight text-to-speech
6. Stable Audio Music/SFX -- Sound effect and music generation
7. ACE-Step Music -- Full song generation with lyrics
8. Video Production Studio -- Create videos with subtitles
9. Chatbot -- AI chat with local or cloud LLMs
10. Master Cheat Sheet -- Quick reference for recommended settings
11. Helpful Links -- External documentation links

All sections start collapsed. Click on any section header to expand it.

The Toolbar

The toolbar sits at the top of the page and contains two important features:

Shutdown Button

The red Shutdown button gracefully stops the server. This properly unloads models and cleans up resources. Always use this instead of just closing the terminal window when possible.

Settings Presets

Next to the shutdown button, you'll find the settings preset controls:

- Preset dropdown -- Select a previously saved preset to load
- Preset name input -- Type a name for a new preset
- Save Settings -- Saves all current parameters across every section into a named preset

This is extremely useful when you've dialed in the perfect combination of parameters for a particular voice or task and want to recall them later.

The Three-Column Layout

Most feature sections follow a consistent three-column layout:

Left Column	Center Column	Right Column
Model controls	Text input / prompts	Output controls
Voice selection	API payload view	Audio format
Device selection		Project save path
Load/Unload buttons		Generate/Stop buttons
Settings gear icon		Results and playback

The Settings Gear

In the left column of most sections, you'll notice a small gear icon. Clicking it slides the panel over to reveal inference settings (temperature, accuracy, etc.). Click the back arrow to return to the main panel. This keeps the interface clean while still giving you full control.

The API Payload Button

In the center column, most sections have an "API Payload" button. Clicking it shows you the exact JSON dictionary that would be sent to the API endpoint. This is useful for:

- Developers who want to call the API programmatically
- Debugging when something isn't working as expected
- Copying the payload to use in scripts or other tools

Device Selection

Every model has a device dropdown that lets you choose where to load it:

Option	Meaning
CPU	Run on your processor (slow but always available)
0	First GPU (usually your primary graphics card)
1	Second GPU
2-10	Additional GPUs if you have them

If you only have one GPU, use 0. If you don't have a GPU, use CPU.

Tip: You can spread models across multiple GPUs. For example, load XTTS on GPU 0 and Whisper on GPU 1 to run them simultaneously without competing for memory.

The Load/Unload Workflow

Every AI model in LocalSoundsAPI follows the same pattern:

1. Select your device from the dropdown (CPU, 0, 1, etc.)
2. Click Load -- The model downloads (first time only) and loads into memory
3. Use the model -- Generate speech, music, transcriptions, etc.
4. Click Unload when done -- Frees the GPU/CPU memory

The status badge next to the Load/Unload buttons shows the current state:

- NOT LOADED (grey) -- Model is not in memory
- LOADED (green) -- Model is ready to use
- LOADING (yellow) -- Model is being loaded

Important: You don't need to unload before switching to another feature. But if you're running low on GPU memory, unloading models you're not currently using is the best way to free space.

Project Save Paths

Several sections include a text input labeled with placeholder text like 'Project', 'Project\SubDirectory'. This controls where generated files are saved:

- Leave it blank -- Files go to the temporary output_tts/ directory (may be cleared on restart)
- Enter a name like MyNovel -- Files are saved to projects_output/MyNovel/
- Enter a path like MyNovel\Chapter1 -- Files are saved to projects_output/MyNovel/Chapter1/

Project folders are persistent and are never automatically deleted.

Output Formats

All TTS and audio generation sections let you choose the output audio format:

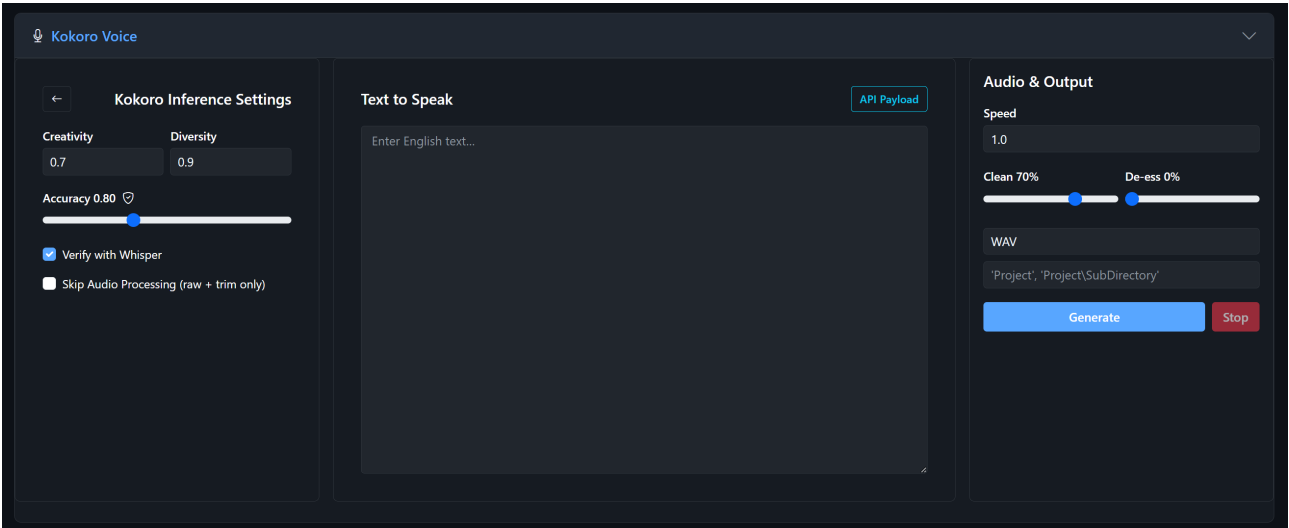
Format	Best For
WAV	Highest quality, no compression, large files
FLAC	Lossless compression, good quality-to-size ratio
MP3	Universal compatibility, lossy compression
OGG	Good quality, smaller than WAV, open format
M4A	Good quality, widely supported on Apple devices

Your First Generation (Quick Start)

Here's the fastest path to hearing your first AI-generated speech:

1. Launch the app and open it in your browser
2. Click "Kokoro Voice" to expand that section (it's the lightest model)
3. In the left column, select a voice from the dropdown (try af_heart)
4. Select device 0 (or CPU) and click Load
5. Wait for the status badge to turn green
6. In the center text area, type: Hello! This is my first time using LocalSoundsAPI.
7. In the right column, click Generate
8. An audio player will appear in the results area -- click play to listen

Congratulations! You've just generated your first AI speech.



The Kokoro Voice section expanded, demonstrating the standard three-column layout: inference settings (left), text input (center), and audio output controls (right).

Chapter 4: Upload and Transcribe

The Upload & Transcribe section handles two important tasks: uploading voice reference files for voice cloning, and transcribing audio to text using OpenAI's Whisper model.

Section Layout

When you expand this section, you'll see three columns:

Left	Center	Right
Whisper Model controls	Transcription text / Audio file list	Upload dropzone

Uploading Voice Reference Files

The right column contains a drag-and-drop upload zone. This is where you upload voice samples that you want to clone with XTTS or Fish Speech.

How to Upload

1. Drag and drop a .wav, .flac, or .mp3 file onto the upload zone, OR
2. Click the upload zone to open a file picker

Uploaded files are saved to the voices/ directory and immediately become available in the voice selection dropdowns of XTTS and Fish Speech.

Tips for Good Voice Reference Files

- Duration: 10-30 seconds of clear speech works best
- Quality: Use clean audio without background music, echo, or noise
- Content: The reference should contain natural, varied speech -- avoid monotone readings
- Format: WAV is preferred for best quality, but MP3 and FLAC work too

Auto Transcribe on Upload

Below the upload zone, there's a checkbox labeled "Auto Transcribe on Upload". When enabled, the app will automatically transcribe your uploaded voice file using Whisper as soon as the upload completes. This is useful for verifying that Whisper can clearly understand the reference audio -- if the transcription looks garbled, the voice sample may not be clean enough for good cloning results.

The Whisper Transcription Model

The left column contains the Whisper model controls.

Loading Whisper

1. Select a device from the dropdown (GPU recommended for speed)
2. Click Load Whisper
3. Wait for the status badge to show "LOADED"

The Whisper model size is configured in config.py:

Model	File	VRAM	Speed	Accuracy
base.en	base.en.pt	~1.5 GB	Very fast	Good
medium.en	medium.en.pt	~5 GB	Moderate	Very good
large-v3	large-v3.pt	~10 GB	Slow	Best

The default is medium.en which gives the best balance of speed and accuracy for English audio.

Unloading Whisper

Click Unload to free the GPU memory when you're done transcribing.

Transcribing Audio

There are two ways to reach transcription:

Method 1: Upload and Auto-Transcribe

1. Enable the "Auto Transcribe on Upload" checkbox
2. Upload a voice file
3. The transcription appears automatically in the center text area

Method 2: Select from Audio Files

1. In the center column, click "Audio Files" to switch to the file list view
2. A dropdown shows all audio files that have been uploaded
3. Select a file and click "Transcribe Selected"
4. Click "Back" to see the transcription text

Transcription Output

The transcription appears in the center column's text area as plain text. Below it, a "Copy to Clipboard" button lets you quickly copy the full transcription.

Behind the scenes, Whisper also generates:

- Word-level timestamps -- stored as a _timing.json file alongside the voice file
- Transcription cache -- if you transcribe the same file again, it returns instantly from cache

When Is Whisper Used?

Whisper is used in several places throughout the app:

1. Voice reference transcription -- As described above
2. TTS verification -- When "Verify with Whisper" is enabled (in XTTS, Fish, or Kokoro settings), the app transcribes each generated audio chunk and compares it to the input text. If the accuracy is below the tolerance threshold, it retries the generation.
3. Video production transcription -- In the Video Production Studio, Whisper generates subtitles (SRT files) from audio
4. Standalone transcription -- You can transcribe any audio file for its text content

Whisper Verification in TTS

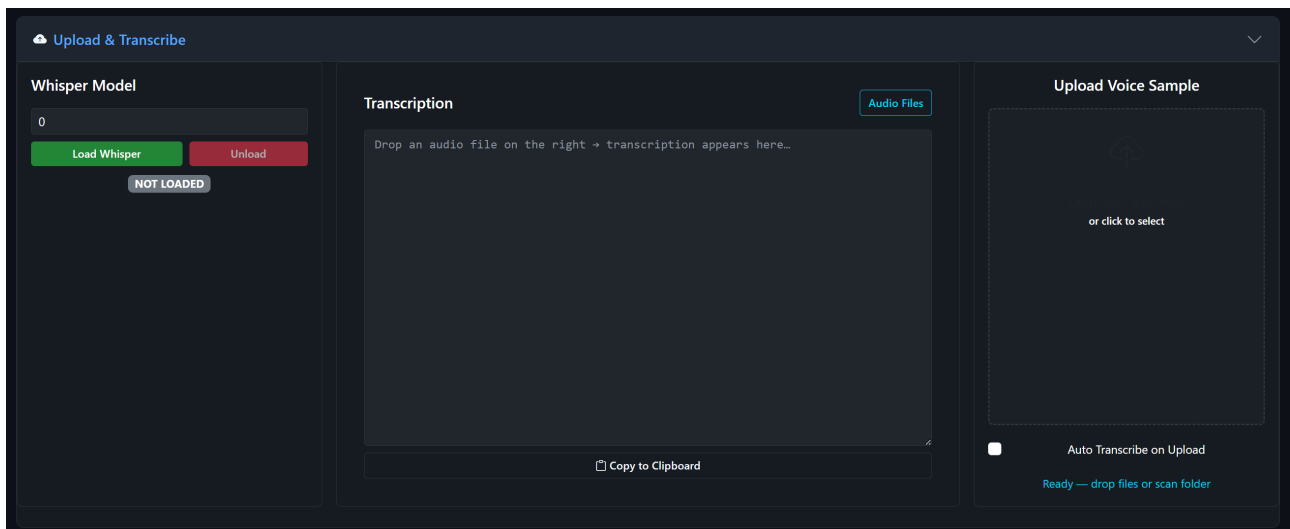
This is one of LocalSoundsAPI's most powerful features. When enabled, after each chunk of text is converted to speech, the app:

1. Runs Whisper on the generated audio
2. Compares the transcription to the original input text
3. Calculates an accuracy score
4. If the score is below the tolerance threshold (e.g., 80%), it retries the generation

This catches garbled audio, skipped words, and hallucinated content automatically. The "Accuracy" slider in each TTS engine's settings panel controls this threshold.

Trade-offs:

- Higher accuracy (85-95%) = Stricter checking, more retries, slower but safer
- Lower accuracy (70-80%) = More forgiving, faster, occasional imperfect output
- Turned off = Fastest, no quality checking, results may vary



The Upload & Transcribe section showing the Whisper model controls (left), transcription text area with Copy to Clipboard button (center), and the voice sample upload dropzone (right).

Chapter 5: XTTS-v2 Voice Cloning

XTTS-v2 (by Coqui AI) is the most versatile text-to-speech engine in LocalSoundsAPI. Its standout feature is voice cloning -- give it a short audio sample of any voice, and it can generate new speech that sounds like that person. It also comes with 59 built-in speaker presets.

When to Use XTTS

- You need to clone a specific voice from a reference recording
- You want multilingual speech generation
- You want a balance between quality and speed
- You want access to a large library of built-in voices

Section Layout

Left Column	Center Column	Right Column
Voice mode (Cloned/Built-in)	Text to speak	Speed control
Voice selection dropdown	API Payload view	Clean (de-reverb) slider
Device selection		De-ess slider
Load/Unload + status		Output format
Gear icon for settings		Project save path
		Generate/Stop buttons
		Results/playback

Loading the Model

1. In the left column, select your device (GPU 0 recommended)
2. Click the green Load button
3. Wait for the status badge to show LOADED (green)

First-time loading downloads the model from Hugging Face (~6 GB).

Voice Modes

At the top of the left column, two buttons let you switch between voice modes:

Cloned Mode

Uses your uploaded voice reference files from the voices/ directory.

1. Click Cloned
2. Select a voice from the dropdown
3. Click Refresh if you've recently uploaded new voices
4. An audio player appears so you can preview the reference voice

Built-in Mode

Uses one of 59 pre-trained speaker voices included with the model.

1. Click Built-in
2. Select a speaker from the dropdown
3. Click Refresh to reload the speaker list

Built-in voices are consistent and reliable. They're a great choice when you don't need a specific cloned voice.

Writing Your Text

In the center column, type or paste the text you want to convert to speech.

Tips for best results:

- Use proper punctuation -- commas, periods, and question marks help the model produce natural pacing
- Break very long text into paragraphs. The app automatically chunks long text into smaller pieces for processing
- Avoid unusual symbols, emojis, or excessive formatting
- Numbers and abbreviations are generally handled well, but spelling them out can improve consistency

Inference Settings (Gear Panel)

Click the gear icon in the left column to reveal the inference settings:

Temperature (default: 0.65)

Controls the randomness of the generated speech.

Value	Effect
0.50-0.60	More predictable, slightly robotic
0.65-0.75	Recommended -- natural and stable
0.80-0.90	More expressive but may slur or wander
> 0.90	Unpredictable, likely to produce artifacts

Repetition Penalty (default: 2.1)

Prevents the model from repeating words or getting stuck in loops. Higher values reduce repetition more aggressively.

Value	Effect
1.0-2.0	Minimal intervention, may stutter on long text
2.0-4.0	Good for most text
4.0-6.0	Aggressive -- use for very long paragraphs

Accuracy / Tolerance (default: 0.80)

When Whisper verification is enabled, this sets the minimum acceptable accuracy score. Generated audio below this threshold is regenerated.

- Drag the slider to adjust (range: 0.70 to 0.98)
- The current value displays next to the label

Verify with Whisper (default: on)

When checked, each generated audio chunk is verified against the original text using Whisper. This catches errors but adds processing time. See Chapter 4 for details.

Skip Audio Processing

When checked, bypasses the post-processing pipeline (normalization, de-reverb, de-ess) and only applies basic trimming. Use this if you plan to do your own audio processing externally.

Audio Output Controls (Right Column)

Speed (default: 1.0)

Controls playback speed. The app uses RubberBand for pitch-corrected speed changes, so the voice pitch stays natural even at different speeds.

Value	Effect
0.5-0.8	Slow, deliberate speech
0.9-1.1	Natural speed
1.2-1.5	Faster narration
1.6-2.0	Very fast, may lose clarity

Clean / De-Reverb (default: 70%)

Reduces room reverb and echo in the generated audio. Higher values apply more aggressive cleaning.

- 0% -- No cleaning, raw output
- 50-80% -- Recommended for most cases
- 100% -- Maximum cleaning, may sound slightly processed

De-Ess (default: 0%)

Reduces harsh "S" and "SH" sounds (sibilance). Only increase this if you notice distracting sibilance.

- 0% -- No de-essing (default)
- 20-50% -- Mild sibilance reduction
- 50-100% -- Aggressive de-essing

Output Format

Choose from WAV, FLAC, MP3, OGG, or M4A.

Save Path

Enter a project folder name (e.g., MyAudiobook) to save output to projects_output/MyAudiobook/. Leave blank for temporary output.

Generating Speech

1. Make sure the model is loaded (green badge)
2. Select your voice (cloned or built-in)
3. Type your text in the center area
4. Adjust output settings as needed
5. Click Generate

The generation status appears below the Generate button, showing progress for multi-chunk text. When complete, an audio player appears in the results area at the bottom of the right column.

Stopping Generation

Click Stop to cancel a generation in progress. Already-completed chunks are preserved.

Job Recovery

If a long generation fails partway through (due to a crash, timeout, or repeated Whisper failures), the app saves a `job.json` file in the project directory. To recover:

1. Type `##recover##` in the text input area
2. Make sure the same project save path is set
3. Click Generate

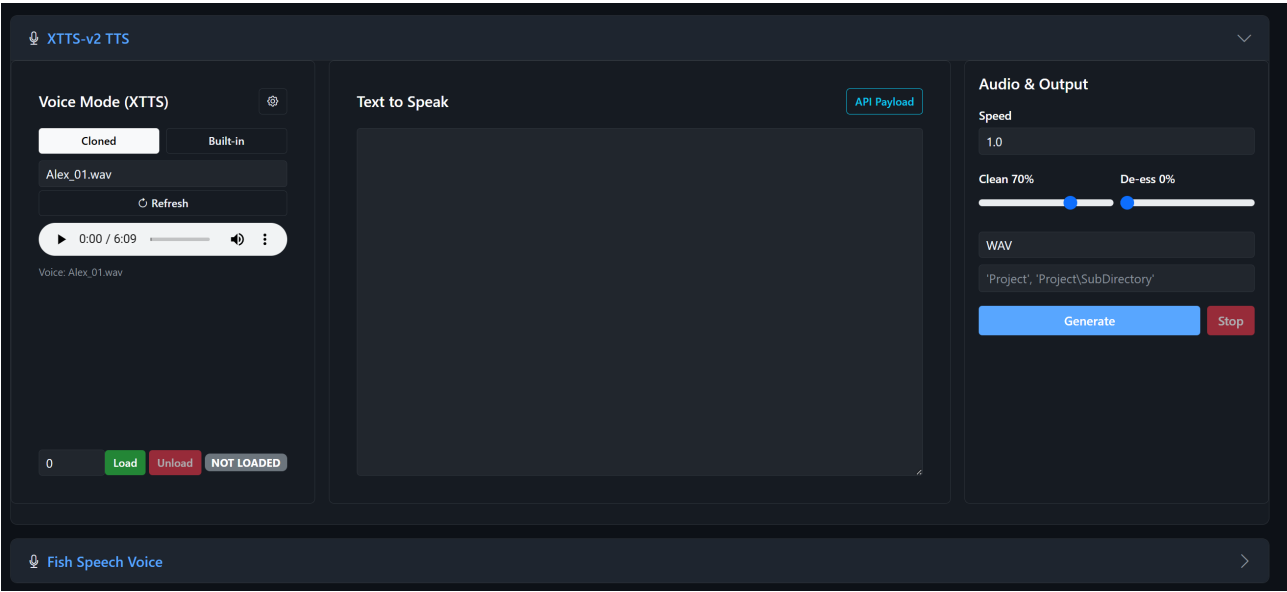
The app reads the `job.json` file and retries only the failed chunks, keeping all the successfully generated chunks intact.

You can also manually edit the `job.json` file to:

- Mark specific chunks as needing retry
- Disable Whisper verification for problematic chunks (`"verify_whisper": false`)
- Adjust other per-chunk parameters

The API Payload

Click "API Payload" in the center column to see the exact dictionary that will be sent to the `/infer` endpoint. You can click "Copy DICT" to copy it for use in scripts or external tools.



The XTTS-v2 section in Cloned voice mode. Left: voice selection with Cloned/Built-in toggle, reference audio preview player, and device/Load controls. Center: text input area with API Payload button. Right: Speed, Clean, De-ess sliders, output format, project path, and Generate/Stop buttons.

Chapter 6: Fish Speech TTS

Fish Speech (OpenAudio S1-Mini) is a high-quality text-to-speech engine that excels at producing natural, realistic speech. While it's slower than XTTS, it often produces superior audio quality, making it the go-to choice when quality matters more than speed.

When to Use Fish Speech

- You need the highest possible voice cloning quality
- You have a GPU with sufficient VRAM (~8 GB)
- You're producing final output where quality justifies longer generation times
- You want near-perfect voice similarity from a reference sample

How Fish Speech Differs from XTTS

Feature	XTTS-v2	Fish Speech
Voice cloning quality	Good	Excellent
Speed	Moderate	Slow (3-8x real-time)
Built-in voices	59 presets	None (reference only)
VRAM required	~6 GB	~8 GB
CPU support	Yes	Yes (very slow)
Multilingual	Yes	Yes
Reference audio	Any length	Best at 8-29 seconds

Section Layout

The layout matches the XTTS section's three-column design:

Left Column	Center Column	Right Column
Voice selection	Text to speak	Speed control
Device selection	API Payload view	Clean / De-reverb
Load/Unload		De-ess
Gear icon for settings		Output format
		Save path
		Generate/Stop
		Results

Loading the Model

1. Select your device (GPU recommended)
2. Click Load
3. First load downloads the model files from Hugging Face

Selecting a Voice

Fish Speech uses your uploaded voice references from the voices/ directory. It does not have built-in speaker presets.

1. Select a voice from the Fish Speech Voice dropdown
2. Click Refresh if you've recently uploaded new voices
3. The audio player lets you preview the reference

Important: Fish Speech automatically trims reference audio to a maximum of 29 seconds. For best results, use clean reference audio between 8 and 20 seconds long.

Inference Settings (Gear Panel)

Click the gear icon to access Fish Speech-specific settings:

Temperature (default: 0.7)

Controls output randomness. Fish Speech is more sensitive to temperature than XTTS.

Value	Effect
0.55-0.65	Stable, controlled output
0.65-0.75	Recommended -- natural with good stability
> 0.75	May cause accent drift or unnatural inflection

Top P (default: 0.7)

Nucleus sampling threshold. Controls the diversity of generated speech.

Value	Effect
0.60-0.70	Focused, predictable
0.70-0.85	Recommended range
> 0.90	More varied but less controlled

Accuracy / Tolerance (default: 0.80)

Same as XTTS -- sets the Whisper verification threshold.

Verify with Whisper (default: on)

Enables quality checking of generated audio.

Skip Audio Processing

Bypasses post-processing for raw output.

Reference Style Prompt (optional)

A text description that guides the speaking style. For example:

- A clear, professional speaker.
- A warm, conversational narrator with a calm tone.

- An energetic announcer with enthusiasm.

This prompt influences how Fish Speech interprets the voice reference and renders the speech.

Audio Output Controls

The right column controls are identical to XTTS:

- Speed -- Playback speed with pitch correction (default: 1.0)
- Clean / De-Reverb -- Room reverb reduction (default: 70%)
- De-Ess -- Sibilance reduction (default: 0%)
- Output Format -- WAV, FLAC, MP3, OGG, M4A
- Save Path -- Project folder name or leave blank for temp

Generating Speech

1. Load the model and select a voice
2. Type your text in the center area
3. Optionally set a reference style prompt in the settings
4. Click Generate
5. Wait for completion -- Fish Speech is slower but produces high-quality output

Generation Status

The status text below the Generate button shows real-time progress:

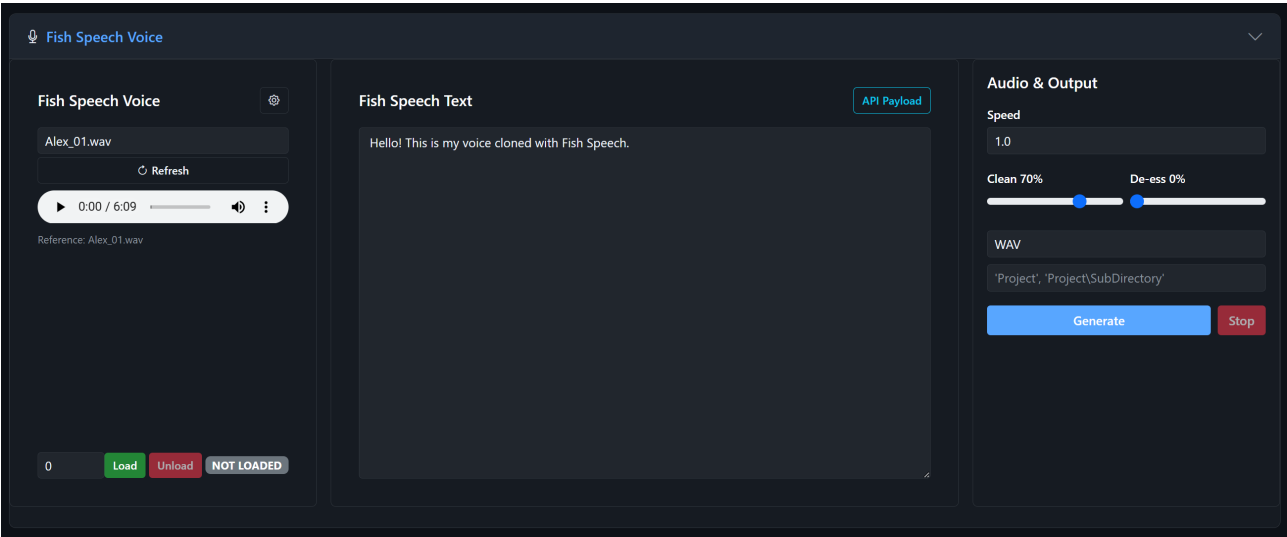
- Which chunk is being processed
- Whether Whisper verification passed or if retries are needed
- Total elapsed time

Stopping and Recovery

Click Stop to cancel. Job recovery works the same as XTTS -- type `##recover##` in the text area with the same save path set to retry failed chunks.

Tips for Best Results

1. Reference audio quality is paramount. Clean, noise-free reference audio between 8-20 seconds produces the best clones.
 2. Keep temperature conservative. Fish Speech can drift with higher temperatures more than other engines.
 3. Use the reference style prompt. A brief style description can significantly improve output consistency.
 4. Allow for longer generation times. Fish Speech prioritizes quality over speed. On an RTX 4090, expect 3-8x real-time (a 10-second clip takes 30-80 seconds to generate).
 5. Monitor Whisper verification. If you see many retries, try lowering the accuracy threshold slightly, or adjust the temperature downward.
-



The Fish Speech section with a voice reference selected (Alex_01.wav) and sample text entered. Left: voice selection with Refresh button, reference audio preview, and device/Load controls. Center: text input area. Right: audio output controls.

Chapter 7: Kokoro TTS

Kokoro is a compact, blazing-fast text-to-speech engine with only 82 million parameters. It's the speed champion of LocalSoundsAPI -- capable of real-time or faster generation on both CPU and GPU. Kokoro comes with 19 high-quality English voices and uses minimal memory.

When to Use Kokoro

- You need fast, real-time speech generation
- You're working on a machine with limited GPU memory
- You want to generate speech on CPU at reasonable speeds
- You need quick previews before committing to a slower, higher-quality engine
- You're building a real-time application that needs low-latency TTS

How Kokoro Compares

Feature	XTTS-v2	Fish Speech	Kokoro
Speed	Moderate	Slow	Very fast
VRAM	~6 GB	~8 GB	~2 GB
CPU performance	Slow	Very slow	30-40x real-time
Voice cloning	Yes	Yes	No (preset voices only)
Voice count	59 built-in + cloned	Cloned only	19 preset voices
Quality	Very good	Excellent	Good

Available Voices

Kokoro includes 19 English voices split into female (af_) and male (am_) prefixes:

Female Voices

Voice ID	Name
af_alloy	Alloy
af_aoele	Aoele
af_bella	Bella
af_heart	Heart
af_jessica	Jessica
af_kore	Kore
af_nicole	Nicole
af_nova	Nova
af_river	River
af_sarah	Sarah
af_sky	Sky

Male Voices

Voice ID	Name
am_adam	Adam
am_echo	Echo
am_eric	Eric
am_fenrir	Fenrir
am_liam	Liam
am_michael	Michael
am_onyx	Onyx
am_puck	Puck

Note: Kokoro does not support voice cloning. You can only use the preset voices listed above.

Section Layout

Same three-column layout as the other TTS engines:

Left Column	Center Column	Right Column
Voice selection	Text to speak	Speed
Device selection	API Payload view	Clean / De-reverb
Load/Unload		De-ess
Gear icon		Output format
		Save path
		Generate/Stop
		Results

Loading the Model

1. Select a voice from the Kokoro Voice dropdown
2. Select your device (CPU works great with Kokoro!)
3. Click Load
4. Wait for the status badge to show LOADED

Requirement: Kokoro requires eSpeak-ng for phoneme processing. Make sure the bin/espeak-ng/ directory contains libespeak-ng.dll and the espeak-ng-data folder. See Chapter 2 for setup details.

Inference Settings (Gear Panel)

Creativity (Temperature) (default: 0.7)

Controls expressiveness and variation.

Value	Effect
0.1-0.5	Very stable, almost monotone
0.5-0.8	Natural expressiveness
0.8-1.0	Very expressive, may be inconsistent
> 1.0	Experimental, often too chaotic

Diversity (Top P) (default: 0.9)

Controls the range of vocal variation.

Value	Effect
0.5-0.7	Predictable output
0.7-0.9	Good variation
> 0.9	Maximum diversity

Accuracy / Tolerance (default: 0.80)

Whisper verification threshold.

Important note for Kokoro: The cheat sheet recommends setting Whisper verification to Off or 92%+ for Kokoro. Lower verification thresholds tend to produce many false failures with Kokoro's output characteristics.

Verify with Whisper (default: on)

Quality checking. Consider turning this off for Kokoro if you experience excessive retries.

Skip Audio Processing

Bypasses normalization and cleaning.

Audio Output Controls

- Speed (default: 1.0) -- Range: 0.5 to 2.0. Kokoro handles speed changes well. The cheat sheet recommends 1.3-1.6 for narration. Above 1.7, voices can sound unnaturally high-pitched ("chipmunk").
- Clean / De-Reverb (default: 70%)
- De-Ess (default: 0%)
- Output Format -- WAV, MP3, OGG, FLAC, M4A
- Save Path -- Project folder or blank for temp

Generating Speech

1. Load the model and select a voice
2. Type your text
3. Click Generate
4. Results appear almost instantly for short text

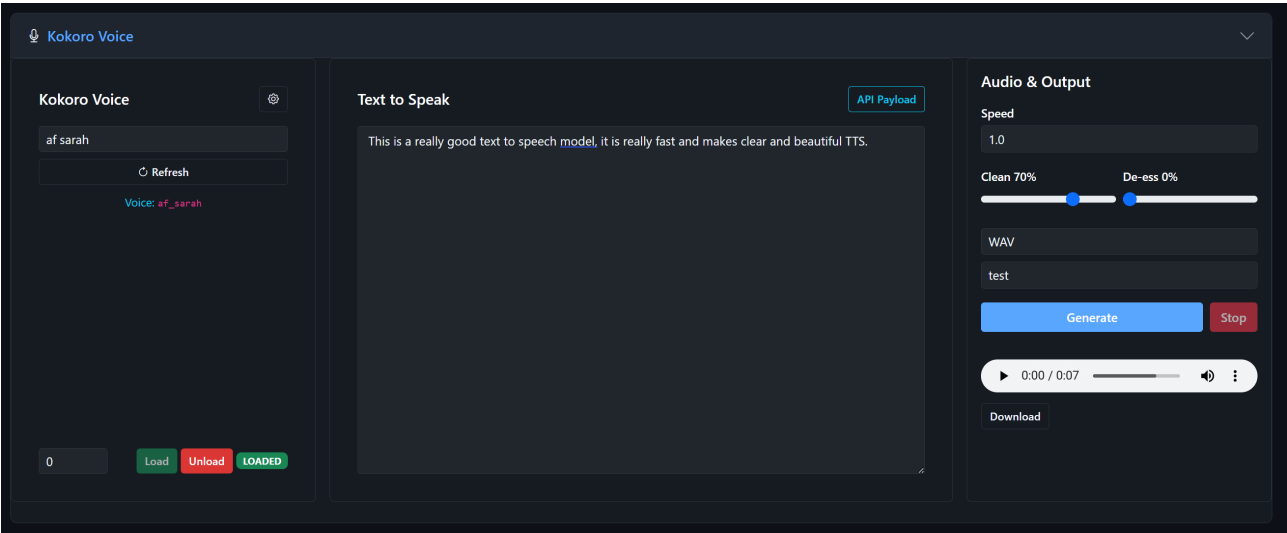
Known Limitation: Unknown Words

Kokoro uses eSpeak-ng for phoneme conversion. If a word is not in eSpeak's dictionary (unusual proper names, made-up words, some technical jargon), Kokoro may:

- Skip the word entirely (silence where the word should be)
- Mispronounce it significantly

Workarounds:

- Spell out phonetically: "Elon" instead of a problematic name
- Use the Whisper verification to catch silent/missing words
- Switch to XTTS or Fish Speech for text with many unusual names



Kokoro Voice with the "af sarah" voice loaded (green LOADED badge). A completed generation shows the audio player in the right column with playback controls and a Download button. Note the fast generation time -- Kokoro is the speed champion.

Chapter 8: Stable Audio -- Music and Sound Effects

Stable Audio (by Stability AI) generates music clips and sound effects from text descriptions. Describe what you want to hear, and the model creates original audio up to 47 seconds long. Combined with the CLAP scoring model, it can generate multiple variants and automatically rank them by quality.

When to Use Stable Audio

- You need sound effects (impacts, ambient sounds, foley, explosions)
- You want short music clips (jingles, intros, background music)
- You need original audio that doesn't come from a sample library
- You want to experiment with creative sound design

Section Layout

Left Column	Center Column	Right Column
Quick prompt templates	Prompt text area	Waveforms count (1-4)
Device selection	Negative prompt	Audio mode
Load/Unload	API Payload view	Output format
Gear icon for settings		Save path
		Generate/Stop
		Results

Loading the Model

1. Select your device (GPU only -- Stable Audio is GPU-intensive)
2. Click Load
3. First load downloads the model (~10 GB VRAM required)

Note: Stable Audio also loads the CLAP model for quality scoring. This adds approximately 1 GB of VRAM.

Quick Prompt Templates

The left column provides one-click prompt templates to get you started:

Template	Description
Deep House	Electronic deep house music with drums and bass
Hip-Hop	Hip-hop beat with samples and rhythm
Cinematic	Orchestral cinematic score
Impact	Punchy impact sound effect
Ambient	Atmospheric ambient soundscape
Explosion	Explosion and blast sound effect
Clear	Clears the prompt fields

These templates fill in both the prompt and appropriate settings. You can use them as-is or modify them as a starting point.

Writing Prompts

Main Prompt

The top text area is where you describe the sound you want. Be specific and descriptive:

Good prompts:

- Warm deep house loop, 120 bpm, rolling bassline, crisp hi-hats, vinyl crackle
- Thunder crack followed by heavy rain on a tin roof, close distance
- Upbeat acoustic guitar strumming, major key, happy summer feeling

Less effective prompts:

- Music (too vague)
- A sound (not descriptive enough)
- Something cool (no useful information for the model)

Negative Prompt

The bottom text area tells the model what to avoid:

- vocals, singing, voice, speech
- distortion, clipping, noise
- drums, percussion (for a drums-free track)

Leave this empty if you don't have specific exclusions.

Audio Modes

The Audio Mode dropdown in the right column controls how the model processes and normalizes the output:

Mode	Best For	Characteristics
Impact / Foley	Short, punchy sound effects	Tight, quick sounds
Ambient / Field	Background textures, loops	Smooth, continuous audio
Jingle / Intro	Music clips	Full, polished sound

Choose the mode that best matches your intended use.

Advanced Settings (Gear Panel)

Click the gear icon to access the diffusion parameters:

Steps (default: 100)

The number of inference steps. More steps = better quality but slower generation.

Value	Effect
10-50	Fast but noisy/low quality
60-80	Decent quality, reasonable speed
80-120	Recommended -- good quality
120-200	Diminishing returns, mostly for experimentation

Length (default: 30.0 seconds)

Duration of the generated audio. Range: 10 to 47 seconds.

- Sound effects: 5-12 seconds usually sufficient
- Music clips: 20-47 seconds for a full passage
- Longer clips with complex prompts may lose coherence

Guidance Scale (default: 7.0)

How closely the model follows your prompt (also called CFG scale).

Value	Effect
1-3	Very loose interpretation
3.5-5.0	Good prompt adherence (recommended for SFX)
5-8	Strong prompt following
8-15	Very strict, may sound forced
> 15	Often over-constrained

Eta (default: 0.0)

Adds controlled noise to the generation process. Usually best left at 0.

Seed (default: -1)

Set a specific seed number for reproducible results. -1 means random. If you get a result you like, note the seed to recreate it later.

Waveform Variants

The Waveforms slider (1 to 4) in the right column controls how many variants are generated per request. The app generates all variants, then uses the CLAP model to score each one against your prompt. Results are displayed ranked from best to worst match.

Recommended: Generate 3-4 variants and pick the best one. Audio generation has inherent randomness, so multiple attempts often yield one standout result.

Generating Audio

1. Load the model
2. Write your prompt (and optionally a negative prompt)
3. Select the audio mode
4. Set the number of waveforms (variants)

5. Click Generate

Results

Each generated variant appears as an audio player in the results area, along with its CLAP quality score. The variants are ranked best-to-worst. You can:

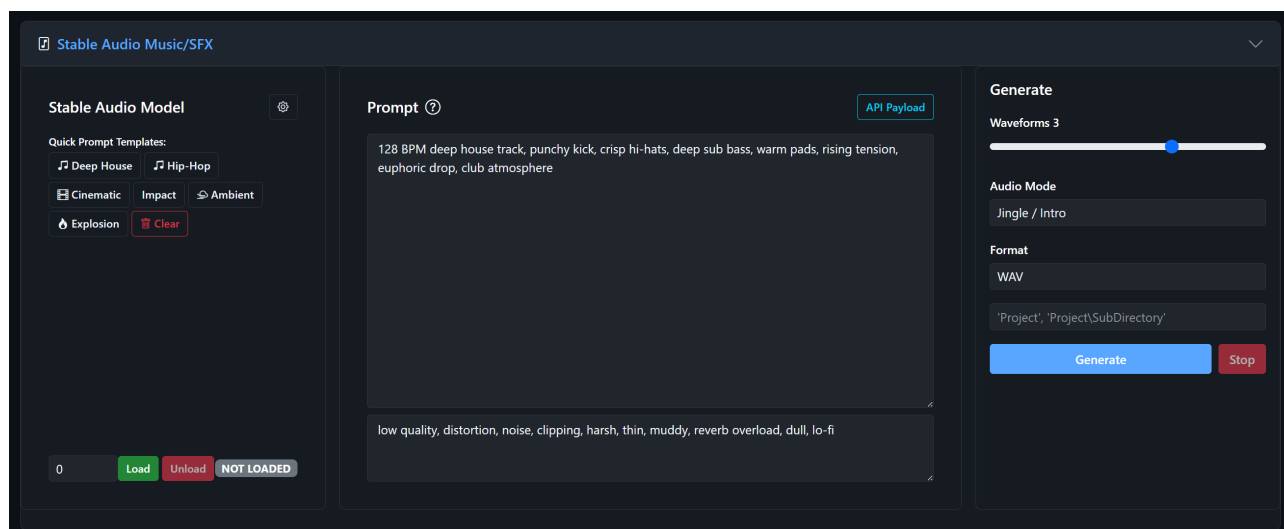
- Play each variant to listen
- Compare scores to identify the best match
- Files are automatically saved if you specified a save path

Stopping Generation

Click Stop to cancel. Completed variants are preserved.

Tips for Better Results

1. Be specific in your prompts. Describe instruments, tempo (BPM), mood, and genre.
2. Use negative prompts. If you keep getting unwanted vocals, add vocals, singing, voice to the negative prompt.
3. Generate multiple variants. The CLAP scoring does a good job of identifying the best outputs.
4. Keep SFX short. Sound effects work best at 5-12 seconds. Longer durations tend to drift.
5. Experiment with guidance scale. Lower values (3-5) give more creative results; higher values (6-8) follow the prompt more closely.
6. Note good seeds. When you find a result you love, remember the seed for reproducibility.



The Stable Audio section showing Quick Prompt Templates (left), prompt and negative prompt text areas (center), and generation controls with Waveforms, Audio Mode, and Format settings (right). A deep house prompt is loaded with genre-specific keywords and a negative prompt filtering out unwanted qualities.

Chapter 9: ACE-Step -- AI Music Generation

ACE-Step is a specialized music generation model designed for creating full-length songs with lyrics, style control, and advanced diffusion parameters. While Stable Audio excels at short clips and sound effects, ACE-Step is purpose-built for music tracks up to 60 seconds.

When to Use ACE-Step

- You want to generate music with lyrics
- You need genre-specific music (EDM, jazz, hip-hop, lo-fi, rock, pop, etc.)
- You're creating longer musical passages (up to 60 seconds)
- You want fine-grained control over the diffusion process

ACE-Step vs. Stable Audio

Feature	Stable Audio	ACE-Step
Max duration	47 seconds	60 seconds
Lyrics support	No	Yes
Genre templates	Basic	Extensive
Sound effects	Excellent	Not designed for SFX
Music quality	Good for clips	Better for songs
VRAM	~10 GB	~7 GB
Advanced controls	Basic	Extensive diffusion params

Section Layout

Left Column	Center Column	Right Column
Genre templates	Prompt (Style + Lyrics)	Variants (1-4)
Device selection	API Payload view	Output format
Load/Unload		Save path
Gear icon for settings		Generate/Stop
		Results

Loading the Model

1. Select your device (GPU required)
2. Click Load
3. First load downloads the ACE-Step model files

Genre Templates

The left column provides quick-start genre templates:

Template	Style
EDM Drop	Electronic dance music with drops and synths
Jazz	Jazz ensemble with improvisation
Lo-Fi	Lo-fi hip-hop / chill beats
Hip-Hop	Hip-hop instrumental
Rock	Rock band arrangement
Pop	Pop production
Rap	Rap beat and flow
R&B	R&B groove
Clear	Clears the prompt

Clicking a template fills the prompt area with an optimized style description and sample lyrics.

Writing Prompts

ACE-Step uses a special prompt format where the first line defines the style and the remaining lines are lyrics:

```
edm drop, 128bpm, supersaw, punchy kick, sidechain, riser
I'm on the edge, feel the drop
Let it go, hands up high
We're flying through the night
Nothing's gonna stop us now
```

Style Line (First Line)

Describe the genre, tempo, instruments, and mood:

- lo-fi hip-hop, 85bpm, dusty vinyl, mellow piano, rain ambiance
- epic cinematic rock, 140bpm, distorted guitars, pounding drums, orchestral strings
- smooth jazz, 100bpm, saxophone solo, walking bass, brush drums

Lyrics (Remaining Lines)

Write the lyrics you want sung or rapped. Keep them simple and rhythmic:

- Short, punchy phrases work better than long sentences
- Match the rhythm to the specified BPM
- Avoid overly complex vocabulary

Instrumental Only

To generate music without vocals, simply provide only the style line with no lyrics, or include instrumental in your style description.

Inference Settings (Gear Panel)

ACE-Step has the most extensive settings of any module. Click the gear icon to access them.

Steps (default: 60)

Diffusion steps. More steps = better quality, slower generation.

Value	Effect
10-30	Fast preview, rough quality
40-60	Recommended -- good quality
60-100	High quality, slower
> 100	Diminishing returns

Duration (default: 10.0 seconds)

Length of the generated music. Range: 1 to 60 seconds.

Guidance (default: 3.5)

Main CFG (classifier-free guidance) scale. Controls how closely the output follows your prompt.

Value	Effect
1-2	Very loose, creative
3-5	Recommended range
5-7	Tight prompt following
> 7	May over-constrain the output

Omega (default: 1.0)

Frequency scaling parameter for the diffusion process.

Min Guidance (default: 1.0)

Minimum guidance value during the generation process. Used with guidance decay.

Guidance Interval (default: 0)

Number of steps at the beginning where full guidance is applied before decaying.

Guidance Decay (default: 1.0)

Rate at which guidance decays over the diffusion steps:

- 1.0 -- No decay (constant guidance)
- < 1.0 -- Guidance decreases over time (more creative ending)
- > 1.0 -- Guidance increases over time (tighter ending)

Guidance Text (default: 0.0)

Additional guidance weight specifically for the text/style component.

Guidance Lyric (default: 0.0)

Additional guidance weight specifically for the lyrics component.

Scheduler

The diffusion scheduler algorithm:

- euler -- Standard Euler method (default, recommended)
- pingpong -- Alternating step sizes, can produce different textures

CFG Type

- `cfg` -- Standard classifier-free guidance
- `ucfg` -- Unconditional classifier-free guidance

ERG Flags

Three checkboxes for ablation experiments:

- `ERG Tag` -- Ablates the tag/style conditioning
- `ERG Lyric` -- Ablates the lyric conditioning
- `ERG Diff` -- Ablates the diffusion guidance

These are advanced experimental options. Leave them unchecked for normal use.

Seed (default: -1)

Random seed. Set a specific number for reproducible results, or -1 for random.

OSS Steps

Optional step schedule override. Enter comma-separated step indices (e.g., 1,3,5) for custom scheduling. Leave blank for default.

Generating Music

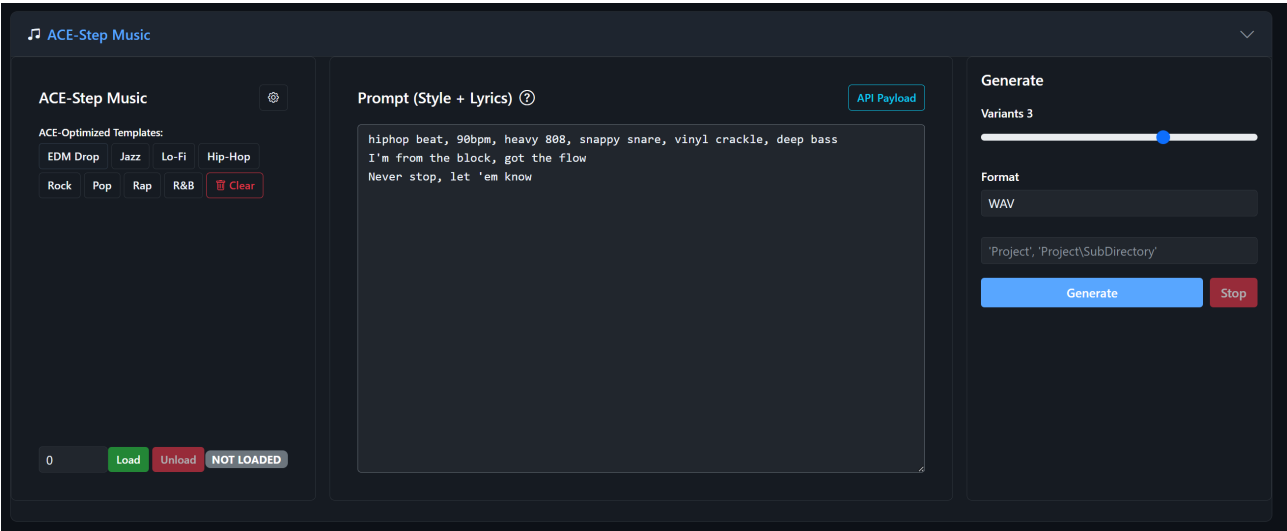
1. Load the model
2. Write your prompt (style line + lyrics) or use a template
3. Set the duration and number of variants
4. Click Generate

Results

Like Stable Audio, each variant is scored by CLAP and displayed ranked. Listen to each variant and choose the best one.

Tips for Better Music

1. Start with templates. The genre templates are tuned for good results. Modify them rather than starting from scratch.
 2. Keep durations reasonable. 10-30 seconds produces the most coherent results. 60 seconds can lose structure.
 3. Match lyrics to tempo. Short, rhythmic phrases at the right syllable count for your BPM produce better singing.
 4. Use 3-4 variants. Music generation has high variance. Multiple attempts ensure at least one good result.
 5. The cheat sheet recommends: Steps 45-55, Guidance 4.5-6.0, first line = genre/style, rest = lyrics.
-



The ACE-Step section with genre template buttons (EDM Drop, Jazz, Lo-Fi, Hip-Hop, Rock, Pop, Rap, R&B) and a hip-hop prompt loaded. The first line defines the style (genre, BPM, instruments) and the remaining lines are lyrics. Right column shows Variants and Format controls.

Chapter 10: Video Production Studio

The Video Production Studio brings together audio, transcription, and video creation in one workflow. Upload audio files, automatically generate subtitles with Whisper, and create videos with various background options and subtitle overlays -- all without leaving the app.

When to Use the Production Studio

- You want to create subtitled videos from audio narration
- You need SRT subtitle files for your audio content
- You want to combine audio with images or video backgrounds
- You're building a pipeline: generate speech -> transcribe -> create video

Section Layout

The Production Studio uses a different layout from the TTS sections. It has four rows:

Row 1: Project Setup

Left Half	Right Half
Video Project Folder selector	File/Folder dropzone

Row 2: Files and Preview

Left Half	Right Half
Project files list	Preview area
Auto-transcribe checkbox	
Auto-create video checkbox	
Transcribe button	

Row 3: Transcription

Full-width transcription text display

Row 4: Video Creation

Full-width video settings and Create Video button

Result Area

Video player with download link

Step 1: Set Up a Project Folder

In the top-left card:

1. Type a project name in the text field (e.g., my_video_project)
 - A simple name creates a folder under projects_output/

- You can also type a full path (e.g., E:\projects\my_video)
- 2. Click Scan to load any existing files in that folder

The "Folder" label is also clickable -- it opens a native folder picker dialog.

Step 2: Upload Media Files

The top-right card contains a drag-and-drop zone. You can:

- Drag audio files (.wav, .mp3, .flac, .ogg, .m4a)
- Drag video files (.mp4, .webm, .mov, .avi, .mkv)
- Drag image files (.jpg, .png, .gif, .bmp, .webp)
- Drag an entire folder of files
- Click the dropzone to use a file picker

Uploaded files are copied into your project folder. The upload status shows progress.

Tip: You don't need to upload files manually if you've been generating TTS with a save path. Files generated by XTTS, Fish Speech, or Kokoro to a project folder are automatically available here.

Step 3: Select and Preview Files

The left half of Row 2 shows a list of all files in your project folder. Click any file to:

- Audio files: Preview with an audio player
- Image files: Preview in the right-side preview area
- Video files: Preview with a video player

Step 4: Transcribe Audio

Before creating a video, you need subtitles. These are generated by transcribing the audio with Whisper.

Prerequisites

Make sure Whisper is loaded (see Chapter 4 -- Upload & Transcribe section, or the Upload & Transcribe panel at the top of the main page).

Manual Transcription

1. Select an audio file from the project files list
2. Click "Transcribe with Whisper"
3. The transcription text appears in Row 3
4. Two files are created alongside the audio:
 - filename.srt -- Subtitle file with timestamps
 - filename_timing.json -- Word-level timing data

Auto-Transcription

Check "Auto-transcribe after upload/select" to automatically transcribe audio files as soon as they're selected or uploaded.

Transcription Caching

If an audio file has already been transcribed (an SRT file exists), the transcription is loaded from cache instantly. You don't need to re-transcribe.

Step 5: Create a Video

Once your audio file has been transcribed (an SRT file exists), you can create a video.

Video Settings (Row 4)

Resolution:

Option	Dimensions	Aspect Ratio
720p	1280 x 720	16:9 (horizontal)
1080p	1920 x 1080	16:9 (horizontal)
1440p	2560 x 1440	16:9 (horizontal)
4K	3840 x 2160	16:9 (horizontal)
1080p Vertical	1080 x 1920	9:16 (vertical/mobile)
4K Vertical	2160 x 3840	9:16 (vertical/mobile)

Background Mode:

Mode	Description
Solid Red Screen	Red background with white subtitles
Solid Green Screen	Green screen for chroma keying
Solid Blue Screen	Blue screen alternative
Cycle Images / Videos	Uses images and videos from the project folder as background
Transparent Background	WebM format with alpha channel transparency

Creating the Video

1. Select the audio file you want to use (must have an SRT subtitle file)
2. Choose your resolution
3. Choose your background mode
4. Click "Create Video"

FFmpeg handles the video creation. This may take a minute or two depending on the audio length and resolution.

Auto-Create Video

Check "Auto-create video after transcription" to automatically start video creation as soon as transcription completes. Combined with auto-transcribe, this creates a fully automated pipeline:

Upload audio -> Auto-transcribe -> Auto-create video

Step 6: View and Download

When the video is complete, a video player appears at the bottom of the section. You can:

- Play the video directly in the browser
- Download using the "Download Video" button

The video file is saved in your project folder.

Background Modes in Detail

Solid Color Screens

Creates a video with a flat color background and subtitle text overlaid. Useful for:

- Red: Default, high contrast with white text
- Green: Green screen for video editing compositing
- Blue: Alternative chroma key color

Cycle Images / Videos

Uses all image and video files in your project folder as rotating backgrounds:

- Images display for a duration based on the audio length divided by the number of images
- Videos play sequentially
- This creates dynamic, visually interesting content automatically

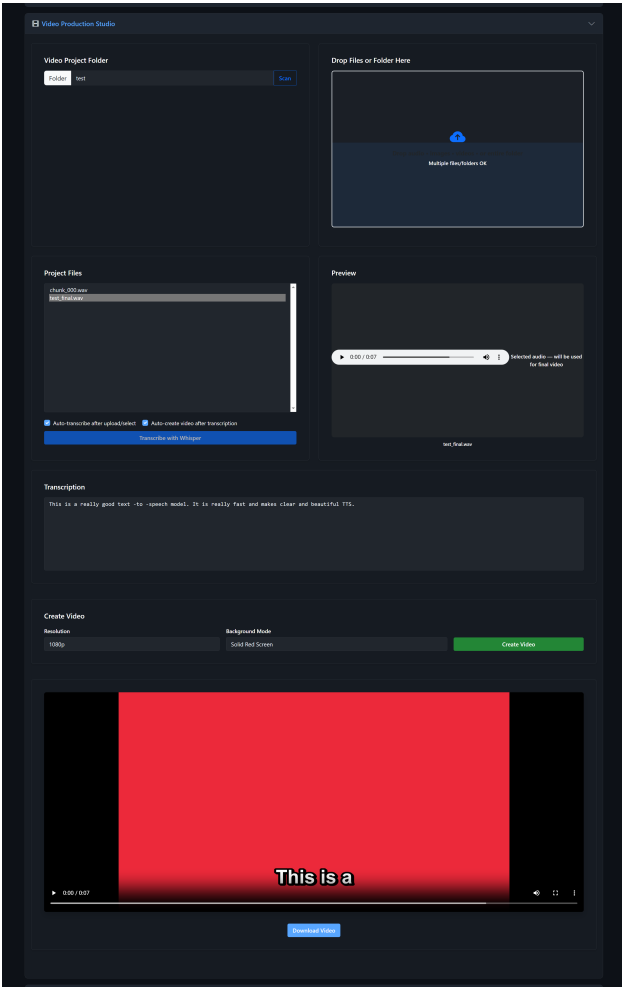
Transparent Background (WebM)

Creates a WebM video with an alpha channel. The background is fully transparent, showing only the subtitle text. This is ideal for overlaying on other video in a video editor.

The Full Production Pipeline

Here's a typical end-to-end workflow combining multiple features:

1. Generate speech using XTTS, Fish, or Kokoro with save path set to MyProject
 2. Open Video Production Studio and set the folder to MyProject
 3. Click Scan -- your generated audio files appear
 4. Upload background images by dragging them into the dropzone
 5. Select an audio file and transcribe it
 6. Set resolution to 1080p, background to "Cycle Images"
 7. Click Create Video
 8. Download the finished video
-



The complete Video Production workflow. Top: project folder and file upload dropzone. Middle: project file list with audio preview, auto-transcribe options, and transcription output. Bottom: video creation settings (1080p, Solid Red Screen) and the finished video with burned-in subtitles playing in the browser.

Chapter 11: Chatbot

The Chatbot section provides an interactive AI conversation interface with three backend options: a local Llama.cpp model, LM Studio integration, and the OpenRouter cloud API. It includes a system prompt manager, conversation history with archiving, and adjustable inference parameters.

Section Layout

Left Column	Center Column	Right Column
LLM Backend selector	Chat history controls	System Prompt manager
Backend-specific controls	Message history display	Preset selector
Model selection	Text input area	Save/load prompts
Load/Unload	Send button	
Gear icon for settings		

Choosing a Backend

The LLM Backend dropdown at the top of the left column lets you switch between three backends:

1. Llama.cpp (Local)

Runs a .gguf model file directly on your machine using llama-cpp-python.

Controls:

- Model dropdown -- Lists all .gguf files found in the directory specified by LLM_DIRECTORY in config.py
- Refresh Models -- Rescans the directory for new model files
- Context Length slider -- How many tokens of conversation history the model can consider (2,048 to 32,768)
- GPU Layers slider -- How many model layers to offload to GPU (0 = pure CPU, 99 = maximum GPU)
- Load / Unload -- Load the selected model or free it from memory

When to use: You want full privacy, have GGUF model files, and want to run everything locally.

2. LM Studio (Proxy)

Connects to a locally-running LM Studio application via its API.

How it works:

- LM Studio runs separately on your machine (download from lmstudio.ai)
- You load and manage models through the LM Studio UI
- LocalSoundsAPI sends chat requests to LM Studio's API endpoint
- No model management needed within LocalSoundsAPI

Controls:

- Status badge showing connection state
- Current model name (whatever's loaded in LM Studio)

When to use: You already use LM Studio and want to use its model management while chatting through LocalSoundsAPI's interface.

Setup: Make sure LMSTUDIO_API_BASE in config.py matches your LM Studio API address (default:

http://127.0.0.1:1234/v1).

3. OpenRouter (Cloud)

Sends requests to the OpenRouter API for cloud-based inference.

Controls:

- Model dropdown -- Lists available models from OpenRouter (Claude, GPT-4, Llama, Mistral, etc.)
- Refresh Models -- Fetches the current model list from OpenRouter

When to use: You want access to the latest large language models without local hardware requirements.

Setup: Set OPENROUTER_API_KEY in config.py with your API key from openrouter.ai. Note: OpenRouter is a paid service.

Privacy note: When using OpenRouter, your conversation is sent to external servers. The local and LM Studio backends keep everything on your machine.

Chatting

Sending Messages

1. Type your message in the text area at the bottom of the center column
2. Press Enter to send (or click the Send button)
3. Use Shift+Enter for a new line without sending

Chat History

The center column displays the full conversation with clear visual distinction between your messages and the AI's responses. The AI's responses stream in real-time, token by token.

History Controls

At the top of the center column, three controls manage your conversation history:

- Clear / Save -- Clears the current conversation and saves it as an archive
- Clear / Delete -- Clears the current conversation without saving
- Load old chat dropdown -- Select a previously archived conversation to restore

Archived conversations are stored as JSON files in brain/context_history/archives/.

Inference Settings (Gear Panel)

Click the gear icon in the left column to access LLM inference parameters:

Temperature (default: 0.80)

Controls randomness in responses.

Value	Effect
0.1-0.3	Very focused, deterministic responses
0.4-0.7	Balanced -- good for factual tasks
0.7-1.0	Creative, varied responses
1.0-2.0	Very creative, potentially incoherent

Max Tokens (default: 8192)

Maximum length of the AI's response in tokens.

Top P (default: 0.95)

Nucleus sampling -- limits the token pool to the most likely options.

Value	Effect
0.5-0.7	Very focused
0.8-0.95	Natural balance
0.95-1.0	Full distribution

Top K (default: 40)

Limits the number of candidate tokens considered at each step.

Presence Penalty (default: 0.0)

Discourages the model from talking about topics it has already mentioned. Range: -2.0 to 2.0.

Frequency Penalty (default: 0.0)

Discourages the model from using words it has already used frequently. Range: -2.0 to 2.0.

System Prompts

The right column is dedicated to system prompt management. The system prompt defines the AI's personality, role, and behavior guidelines.

The Default Prompt

Out of the box, the system prompt is: You are a helpful assistant.

Creating Custom Prompts

1. Type your system prompt in the large text area
2. Enter a preset name in the name field above
3. Click Save Preset

Your prompt is saved to brain/ as a JSON file and appears in the preset dropdown.

Loading a Saved Prompt

1. Select a prompt from the preset dropdown
2. The text area updates with the saved prompt content
3. Changes apply instantly to the current conversation

Prompt Ideas

- Technical assistant: You are a senior software engineer. Answer questions with code examples and technical accuracy.
- Creative writer: You are a creative writing assistant. Help brainstorm ideas, develop characters, and refine prose.
- Tutor: You are a patient tutor who explains concepts step by step, checking understanding at each stage.

- Character roleplay: You are [character name] from [setting]. Stay in character and respond as they would.

Where Prompts Are Stored

- Current active prompt: brain/system_prompt.json
- Saved presets: brain/[preset_name].json
- Conversation history: brain/context_history/current.json
- Archives: brain/context_history/archives/

Conversation Memory

The chatbot automatically maintains conversation history in brain/context_history/current.json. This means:

- Your conversation persists between page refreshes
- The AI remembers earlier messages in the current session
- Context length limits how far back the AI can "see"

Managing Context Length

For local Llama models, the Context Length slider determines how much history is available:

Context	Approximate History
2,048	~1-2 pages of conversation
8,192	~4-6 pages (default)
16,384	~8-12 pages
32,768	~16-24 pages

Larger context uses more memory but lets the AI reference older messages.

Typical Workflows

Quick Q&A

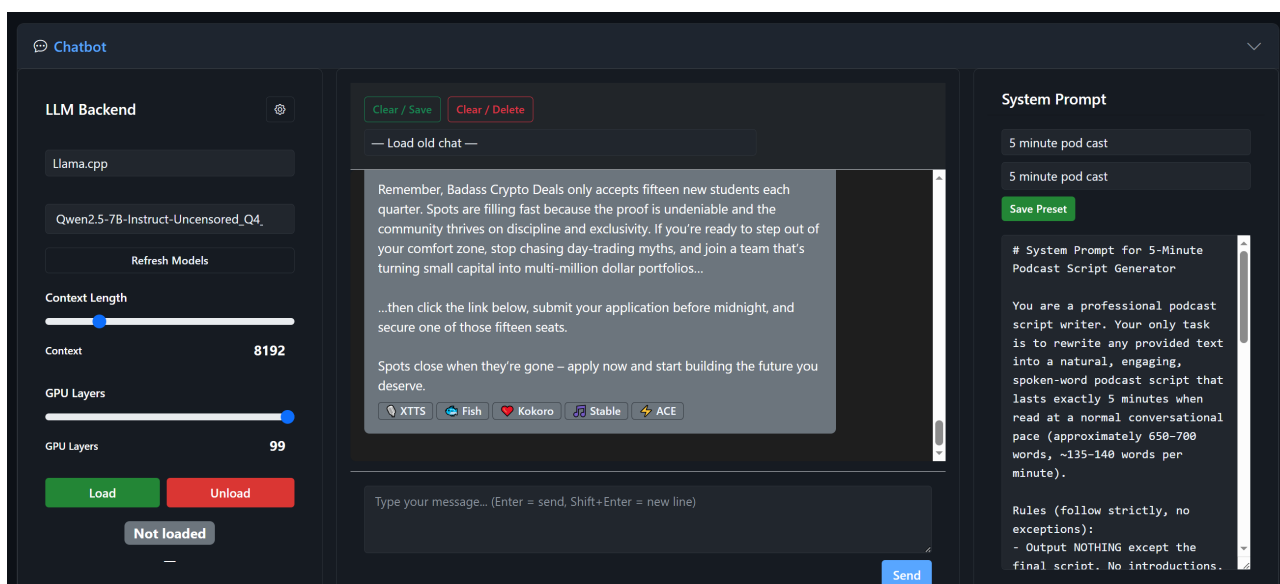
1. Select LM Studio or OpenRouter backend
2. Type a question and press Enter
3. Get a response

Extended Conversation with Memory

1. Load a local Llama model with 16K+ context
2. Set a custom system prompt for your use case
3. Chat naturally -- the model remembers the full conversation
4. When done, click Clear / Save to archive for later reference

Research Across Sessions

1. Start a conversation about a topic
2. Archive it when done
3. Later, load the archive from the dropdown to continue where you left off



The Chatbot with the Llama.cpp backend selected. Left: model selection (Qwen2.5-7B), context length slider, GPU layers control, and Load/Unload buttons. Center: conversation history with Clear/Save and Load old chat controls, plus the message input. Right: system prompt editor with a custom podcast script generator prompt and preset management.

Chapter 12: Settings and Presets

LocalSoundsAPI provides a settings preset system that lets you save and recall the complete set of parameters across all sections. This is invaluable when you've found the perfect combination of settings for a particular voice, genre, or workflow.

The Settings Toolbar

At the top of the main page, the toolbar contains the preset controls:

Control	Purpose
Preset dropdown	Select a saved preset to load
Preset name input	Name for a new preset
Save Settings button	Save current settings to the named preset

Saving a Preset

1. Configure all sections to your liking (temperature, speed, voice selections, audio modes, etc.)
2. Type a name in the preset name input (e.g., Audiobook_Voice_A)
3. Click Save Settings

The preset saves all parameters across every section of the app, including:

- XTTS settings (temperature, repetition penalty, speed, de-reverb, de-ess, voice mode, output format, etc.)
- Fish Speech settings (temperature, top P, speed, reference prompt, etc.)
- Kokoro settings (creativity, diversity, speed, etc.)
- Stable Audio settings (steps, length, guidance, audio mode, etc.)
- ACE-Step settings (steps, duration, guidance, scheduler, all advanced parameters, etc.)
- Chatbot settings (temperature, max tokens, top P, top K, penalties, etc.)

Loading a Preset

1. Click the preset dropdown
2. Select the preset you want
3. All settings across the app update instantly

Default Preset

The A_default preset is included with the app and contains sensible starting values. You can use it as a baseline and create your own presets from there.

Where Presets Are Stored

Presets are saved as JSON files in the settings/ directory:

```
settings/
  A_default.json    -- The built-in default preset
  AA_notes.json     -- Notes about settings (reference file)
  My_Custom.json    -- Your saved presets
```

You can manually edit these JSON files, copy them between machines, or share them with others.

Configuration File (config.py)

For deeper configuration that goes beyond the web UI presets, edit config.py directly. This file controls:

Audio Post-Processing Parameters

Each TTS engine has its own set of post-processing parameters:

XTTS Post-Processing:

```
XTTS_PADDING_SECONDS    = 0.5    -- Silence padding added to chunks
XTTS_CLIPPING_THRESHOLD = 0.95   -- Peak level that triggers clipping protection
XTTS_TARGET_LUFS        = -23.0  -- Loudness normalization target
XTTS_TRIM_DB            = -35     -- Silence detection threshold for trimming
XTTS_MIN_SILENCE        = 500    -- Minimum silence duration (ms) to trim
XTTS_FRONT_PROTECT      = 100    -- Protected silence at start (ms)
XTTS_END_PROTECT        = 800    -- Protected silence at end (ms)
XTTS_FRONT_PAD          = 0.0    -- Additional padding at start (seconds)
XTTS_INTER_PAUSE        = 0.25   -- Pause between chunks (seconds)
```

Fish Speech Post-Processing:

```
FISH_PADDING_SECONDS    = 0.5
FISH_CLIPPING_THRESHOLD = 0.95
FISH_TARGET_LUFS        = -23.0
FISH_TRIM_DB            = -40
FISH_MIN_SILENCE        = 400
FISH_FRONT_PROTECT      = 80
FISH_END_PROTECT        = 600
FISH_FRONT_PAD          = 0.0
FISH_INTER_PAUSE        = 0.2
```

Kokoro Post-Processing:

```
KOKORO_PADDING_SECONDS  = 0.5
KOKORO_CLIPPING_THRESHOLD = 0.95
KOKORO_TARGET_LUFS      = -23.0
KOKORO_TRIM_DB          = -40
KOKORO_MIN_SILENCE      = 300
KOKORO_FRONT_PROTECT    = 250
KOKORO_END_PROTECT      = 1100
KOKORO_FRONT_PAD        = 0.15
KOKORO_INTER_PAUSE      = 0.3
```

Understanding Post-Processing Settings

TARGET_LUFS -- The loudness normalization target. Standard broadcast is -23.0 LUFS. For Kokoro, some voices sound better with different targets:

- -26.0 -- Recommended for most Kokoro voices (am_onyx, af_bella, etc.)
- -25.0 -- Slightly cooler voices (am_adam, af_emma)
- -23.0 -- Default, used for older voice styles

TRIM_DB -- How quiet audio must be to be considered "silence" and trimmed. Lower values (more negative) are more

aggressive.

MIN_SILENCE -- How long a quiet section must be before it's trimmed (in milliseconds).

FRONT_PROTECT / END_PROTECT -- How much silence at the beginning and end of each chunk is protected from trimming (in milliseconds). This prevents cutting off the start or tail of words.

INTER_PAUSE -- How much silence is inserted between concatenated chunks (in seconds). Increase this if words between chunks feel too close together.

Job Recovery Attempts

```
XTTS_AUTO_TRIGGER_JOB_RECOVERY_ATTEMPTS = 3
FISH_AUTO_TRIGGER_JOB_RECOVERY_ATTEMPTS = 3
KOKORO_AUTO_TRIGGER_JOB_RECOVERY_ATTEMPTS = 3
```

When a TTS chunk fails Whisper verification, the system retries up to this many times before marking the chunk as failed. Three attempts is the default and is usually sufficient.

Other Config Options

- DELETE_OUTPUT_ON_STARTUP -- Set to True to clear the temporary output_tts/ folder each time the app starts. Set to False to preserve temporary files.
- WHISPER_PATH -- Choose which Whisper model to use (base.en, medium.en, or large-v3)
- LLM_DEVICE -- GPU device for local Llama models
- LLM_DIRECTORY -- Path to your .gguf model files
- LMSTUDIO_API_BASE -- LM Studio API endpoint
- OPENROUTER_API_KEY -- Your OpenRouter API key

Tips for Managing Settings

1. Create presets for each use case. An "Audiobook" preset, a "Quick Preview" preset, a "Sound Effects" preset, etc.
2. Name presets descriptively. Audiobook_VoiceA_Slow is more useful than preset1.
3. Keep the default preset. Don't overwrite A_default.json -- use it as a reference for starting values.
4. Back up your settings folder. Copy the settings/ directory to keep your presets safe.
5. Share presets. JSON files can be shared with other LocalSoundsAPI users. Just copy the files into their settings/ directory.

Chapter 13: Recommended Settings Cheat Sheet

This chapter reproduces and expands on the "Master Cheat Sheet" built into the app. Use it as a quick reference for dialing in good parameters.

XTTS-v2 -- Daily Driver

Best for: General-purpose TTS, voice cloning, multilingual content

Parameter	Recommended	Notes
Temperature	0.65 - 0.75	Above 0.85 causes slurring; below 0.55 sounds robotic
Speed	0.90 - 1.15	RubberBand handles pitch correction at extreme values
Repetition Penalty	3.0 - 6.0	Too low causes stuttering on long text
Top P / Top K	Default	Only change if you hear gibberish
Whisper Tolerance	78% - 84%	Above 90% is too strict; below 70% lets bad output through
Clean (De-Reverb)	60% - 80%	Good for removing room sound
De-Ess	0% - 30%	Only increase if sibilance is distracting

Strengths: Fast, works on CPU, any language, reliable

Weaknesses: Voice cloning is good but not perfect

Fish Speech -- Highest Quality Cloning

Best for: Premium voice cloning, final production output

Parameter	Recommended	Notes
Temperature	0.60 - 0.70	Above 0.75 causes accent drift
Top P	0.80 - 0.90	Avoid below 0.70
Reference audio	8-20 seconds, clean	No background music or noise
Whisper Tolerance	70% - 80%	Can be stricter than XTTS
Reference Style Prompt	Short, descriptive	"A clear, professional speaker with calm tone"

Strengths: Near-perfect voice cloning

Weaknesses: 3-8x real-time on RTX 4090, occasional failures (use ##recover##)

Kokoro 82M -- Fast CPU Inference

Best for: Quick previews, real-time applications, low-VRAM systems

Parameter	Recommended	Notes
Speed	1.3 - 1.6	Above 1.7 sounds like a chipmunk
Creativity (Temp)	0.5 - 0.8	Keep moderate for natural output
Diversity (Top P)	0.7 - 0.9	Standard range
Whisper Verify	Off or 92%+	Lower thresholds cause many false failures

Strengths: 30-40x real-time on CPU, minimal VRAM

Weaknesses: Unknown words may produce silence

Stable Audio Open 1.0 -- SFX and Short Music

Best for: Sound effects, ambient textures, musical clips up to 47 seconds

Parameter	Recommended	Notes
Length	5-12s (SFX), 30-47s (music)	Long complex prompts get muddy
Steps	80 - 100	Below 60 is noisy
CFG / Guidance Scale	3.5 - 5.0	Higher = better prompt adherence
Waveforms (batch)	3 - 4	Use CLAP scoring to pick the best result
Eta	0.0	Usually best left at zero

Strengths: Excellent sound effects, good music clips

Weaknesses: Max 47 seconds, no vocals

ACE-Step -- Full-Length Songs

Best for: Music with lyrics, genre-specific tracks, longer compositions

Parameter	Recommended	Notes
Steps	45 - 55	Higher gives minimal quality improvement
Guidance	4.5 - 6.0	Too low = incoherent output
Duration	10 - 30 seconds	60s possible but may lose structure
Prompt format	First line = genre/style, rest = lyrics	No special tags needed
Variants	3 - 4	High variance, pick the best

Scheduler	euler	Default, reliable
CFG Type	cfg	Standard guidance

Strengths: Lyrics-aware generation, genre flexibility

Weaknesses: Quality acceptable but not studio-grade

Golden Rules and Essential Tips

These tips apply across all modules:

1. Save path blank = preview only. No file is permanently saved.
 2. Enter a folder name = auto-save. Files go to projects_output/YourFolder/.
 3. `##recover##` = instant job resume. Type this in the text area with the same save path to retry failed chunks.
 4. Whisper keeps failing? Edit the job.json file in your project directory and set "verify_whisper": false for the problematic chunk.
 5. Kill a stuck job. Close the app entirely (use the Shutdown button or close the terminal).
 6. Need true parallel processing? Run multiple instances on different ports (5006, 5007, 5008...).
 7. Post-processing matters. For professional results, run your output through external tools (RubberBand, EQ, compression) after generation.
 8. Unload models when switching tasks. Free GPU memory for the next model.
 9. Use GPU for quality, CPU for previews. GPU is dramatically faster for all models except Kokoro (which runs great on CPU).
 10. Back up your projects folder. projects_output/ contains all your saved work.
-

Chapter 14: API Reference for Developers

LocalSoundsAPI exposes a full REST API that you can call from scripts, other applications, or tools like curl and Postman. Every action available in the web UI has a corresponding API endpoint. All endpoints accept and return JSON unless noted otherwise.

Base URL: `http://127.0.0.1:5006` (or whatever port you configured)

Model Management

These endpoints load and unload AI models. Models must be loaded before inference.

XTTS-v2

Method	Endpoint	Description
POST	<code>/load</code>	Load XTTS model. Body: <code>{"device": "0"}</code>
POST	<code>/unload</code>	Unload XTTS model
GET	<code>/status</code>	Returns model status

Fish Speech

Method	Endpoint	Description
POST	<code>/fish_load</code>	Load Fish Speech. Body: <code>{"device": "0"}</code>
POST	<code>/fish_unload</code>	Unload Fish Speech
GET	<code>/fish_status</code>	Returns model status

Kokoro

Method	Endpoint	Description
POST	<code>/kokoro_load</code>	Load Kokoro. Body: <code>{"device": "0"}</code>
POST	<code>/kokoro_unload</code>	Unload Kokoro
GET	<code>/kokoro_status</code>	Returns model status

Whisper

Method	Endpoint	Description
POST	<code>/whisper_load</code>	Load Whisper. Body: <code>{"device": "0"}</code>
POST	<code>/whisper_unload</code>	Unload Whisper
GET	<code>/whisper_status</code>	Returns model status

Stable Audio

Method	Endpoint	Description
POST	/stable_load	Load Stable Audio. Body: {"device": "0"}
POST	/stable_unload	Unload Stable Audio
GET	/stable_status	Returns model status

ACE-Step

Method	Endpoint	Description
POST	/ace_load	Load ACE-Step. Body: {"device": "0"}
POST	/ace_unload	Unload ACE-Step
GET	/ace_status	Returns model status

Text-to-Speech Inference

XTTS Inference

POST /infer

Generate speech with XTTS-v2.

Request body:

```
{
  "text": "Hello, this is a test.",
  "voice": "my_voice.wav",
  "speaker": null,
  "temperature": 0.65,
  "repetition_penalty": 2.1,
  "speed": 1.0,
  "de_reverb": 70,
  "de_ess": 0,
  "output_format": "wav",
  "save_path": "MyProject",
  "tolerance": 0.80,
  "verify_whisper": true,
  "skip_post_process": false
}
```

Parameters:

- text (required) -- Text to convert to speech. Use `##recover##` to retry failed chunks.
- voice -- Filename of voice reference (from voices/ directory). Used in "Cloned" mode.
- speaker -- Name of built-in speaker. Used in "Built-in" mode. Set voice to null when using this.
- temperature -- Inference temperature (0.1 - 1.0)
- repetition_penalty -- Repetition penalty (1.0 - 6.0)
- speed -- Playback speed (0.5 - 2.0)
- de_reverb -- De-reverb percentage (0 - 100)
- de_ess -- De-ess percentage (0 - 100)
- output_format -- "wav", "mp3", "ogg", "flac", or "m4a"
- save_path -- Project folder name (empty string for temp output)
- tolerance -- Whisper verification threshold (0.70 - 0.98)

- verify_whisper -- Enable/disable Whisper quality checking
- skip_post_process -- Skip audio post-processing

Response: Streaming JSON with chunk progress and final audio file path.

Cancel: POST /xtts_cancel

Fish Speech Inference

POST /fish_infer

Request body:

```
{
  "text": "Hello, this is a test.",
  "voice": "my_voice.wav",
  "temperature": 0.7,
  "top_p": 0.7,
  "speed": 1.0,
  "de_reverb": 70,
  "de_ess": 0,
  "output_format": "wav",
  "save_path": "MyProject",
  "tolerance": 0.80,
  "verify_whisper": true,
  "skip_post_process": false,
  "ref_text": "A clear, professional speaker."
}
```

Additional parameter:

- ref_text -- Reference style prompt describing the desired speaking style

Cancel: POST /fish_cancel

Kokoro Inference

POST /kokoro_infer

Request body:

```
{
  "text": "Hello, this is a test.",
  "voice": "af_heart",
  "temperature": 0.7,
  "top_p": 0.9,
  "speed": 1.0,
  "de_reverb": 70,
  "de_ess": 0,
  "output_format": "wav",
  "save_path": "MyProject",
  "tolerance": 0.80,
  "verify_whisper": true,
  "skip_post_process": false
}
```

Note: The voice parameter is a Kokoro voice ID (e.g., af_heart, am_onyx), not a filename.

Cancel: POST /kokoro_cancel

Audio Generation

Stable Audio Inference

POST /stable_infer

Request body:

```
{
  "prompt": "Ambient forest sounds with water flowing over rocks",
  "negative_prompt": "vocals, speech, noise",
  "steps": 100,
  "length": 30.0,
  "guidance_scale": 7.0,
  "eta": 0.0,
  "seed": -1,
  "num_waveforms": 3,
  "audio_mode": "sfx_ambient",
  "output_format": "wav",
  "save_path": "MyProject"
}
```

Parameters:

- prompt (required) -- Text description of the desired audio
- negative_prompt -- What to avoid
- steps -- Diffusion steps (10-200)
- length -- Duration in seconds (10-47)
- guidance_scale -- CFG strength (1-20)
- eta -- Noise parameter (0-1)
- seed -- Random seed (-1 for random)
- num_waveforms -- Number of variants (1-4)
- audio_mode -- "sfx_impact", "sfx_ambient", or "music"
- output_format -- Output audio format
- save_path -- Project folder

Cancel: POST /stable_cancel

ACE-Step Music Inference

POST /ace_infer

Request body:

```
{
  "prompt": "lo-fi hip-hop, 85bpm, dusty vinyl\nRelaxing in the afternoon\nWatching clouds float by",
  "steps": 60,
  "duration": 10.0,
  "guidance_scale": 3.5,
  "omega": 1.0,
  "min_guidance_scale": 1.0,
  "guidance_interval": 0,
  "guidance_decay": 1.0,
  "guidance_scale_text": 0.0,
  "guidance_scale_lyric": 0.0,
  "scheduler_type": "euler",
  "cfg_type": "cfg",
}
```

```
"erg_tag": false,
"erg_lyric": false,
"erg_diffusion": false,
"seed": -1,
"oss_steps": "",
"num_waveforms": 3,
"output_format": "wav",
"save_path": "MyProject"
}
```

Note: The prompt format uses \n to separate the style line from lyrics.

Voice Management

Method	Endpoint	Description
GET	/voices	List all reference voice files in voices/
GET	/speakers	List all XTTS built-in speaker names
POST	/upload	Upload a voice reference file (multipart form)
POST	/refresh_voices	Refresh the voice file list

Upload Example (curl)

```
curl -X POST http://127.0.0.1:5006/upload \
-F "file=@my_voice.wav"
```

Transcription

Method	Endpoint	Description
POST	/production/transcribe	Transcribe audio file with Whisper
POST	/production/transcribe_status	Check if transcription exists (cache)
POST	/voice_transcribe	Transcribe a voice reference file
POST	/voice_transcribe_cache	Check voice transcription cache
POST	/voice_transcribe_cancel	Cancel voice transcription

Video Production

Method	Endpoint	Description
POST	/production/upload_media	Upload media files to a project folder
GET	/production/list_audio	List audio files. Query: ?dir=MyProject
GET	/production/list_images	List image/video files. Query: ?dir=MyProject
POST	/production/make_video	Create a video with subtitles

Make Video Request

```
{
  "audio_file": "narration.wav",
  "project_dir": "MyProject",
  "resolution": "1080p",
  "bg_mode": "color|green"
}
```

bg_mode options: "color|red", "color|green", "color|blue", "images", "transparent"

Chatbot / LLM

Local Llama.cpp

Method	Endpoint	Description
POST	/chatbot/load	Load a GGUF model
POST	/chatbot/unload	Unload the model
GET	/chatbot/status	Model status
POST	/chatbot/infer	Chat inference (streaming)

LM Studio

Method	Endpoint	Description
POST	/lmstudio/infer	Chat via LM Studio
GET	/lmstudio/status	Connection status
GET	/lmstudio/models	Available models

OpenRouter

Method	Endpoint	Description
POST	/openrouter/infer	Chat via OpenRouter
GET	/openrouter/status	API key status
GET	/openrouter/models	Available models

Brain / Memory

Method	Endpoint	Description
GET	/chatbot/brain/system_prompt	Get current system prompt
POST	/chatbot/brain/system_prompt	Update system prompt
DELETE	/chatbot/brain/system_prompt	Delete a custom prompt
GET	/chatbot/brain/list_system_prompts	List all saved prompts
GET	/chatbot/brain/history	Get conversation history
POST	/chatbot/brain/history	Save conversation history
POST	/chatbot/brain/save_archive	Archive current conversation
GET	/chatbot/brain/list_archives	List archived conversations
POST	/chatbot/brain/load_archive	Load an archived conversation

Settings Presets

Method	Endpoint	Description
GET	/settings/list	List all saved presets
POST	/settings/save	Save a preset. Body: {"name": "MyPreset", "settings": {...}}
POST	/settings/load	Load a preset. Body: {"name": "MyPreset"}
POST	/settings/delete	Delete a preset. Body: {"name": "MyPreset"}

File Serving

Method	Endpoint	Description
GET	/file/	Serve a file. Optional query: ?rel=path/to/file
GET	/audio/	Serve from output_tts or voices directory

Administration

Method	Endpoint	Description
POST	/shutdown	Graceful server shutdown with cleanup

Using the API Payload Feature

Every TTS and audio section in the web UI has an "API Payload" button that shows the exact dictionary that will be sent to the endpoint. This is the easiest way to discover the correct parameters for any API call:

1. Configure the settings in the web UI
 2. Click "API Payload"
 3. Click "Copy DICT"
 4. Use the copied payload in your script
-

Chapter 15: Troubleshooting

Model Loading Issues

Model won't load -- "Out of memory" error

Cause: Not enough GPU VRAM to fit the model.

Solutions:

1. Unload other models first to free VRAM
2. Check available VRAM: open a terminal and run `nvidia-smi`
3. Try loading on a different GPU (if you have multiple)
4. Use CPU mode (slower but always works)
5. For Whisper, switch to a smaller model in `config.py` (`base.en` instead of `medium.en`)

Model won't load -- download fails

Cause: Network issue preventing model download from Hugging Face.

Solutions:

1. Check your internet connection
2. If behind a proxy, configure Python's proxy settings
3. Try again -- Hugging Face may be temporarily unavailable
4. Manually download model files and place them in the correct `models/` subdirectory

Status badge stuck on "LOADING"

Cause: Model loading is taking a long time or has hung.

Solutions:

1. Wait longer -- first-time loads include downloads which can be slow
2. Check the terminal for error messages
3. Restart the app and try again

TTS Generation Issues

Generated speech sounds garbled or slurred

Cause: Temperature too high, or poor voice reference quality.

Solutions:

1. Lower the temperature (try 0.60-0.70)
2. Increase repetition penalty (try 3.0-5.0 for XTTS)
3. Use a cleaner voice reference (no background noise, no music)
4. Try a shorter text -- break long text into smaller pieces

Words are skipped or missing in the output

Cause: Common with Kokoro when encountering unknown words; or Whisper verification is rejecting chunks.

Solutions:

1. For Kokoro: spell out unusual words phonetically or switch to XTTS/Fish
2. Lower the Whisper tolerance threshold
3. Disable Whisper verification temporarily to see if the raw output is acceptable
4. Check the terminal for retry messages

Audio has echo or room reverb

Cause: Voice reference has room reverb, or de-reverb is too low.

Solutions:

1. Increase the Clean/De-Reverb slider (try 80-100%)
2. Use a better voice reference recorded in a quiet, treated room
3. Re-record the reference closer to the microphone

Audio has harsh "S" sounds (sibilance)

Cause: Some voices and TTS engines produce prominent sibilance.

Solutions:

1. Increase the De-Ess slider (try 30-60%)
2. Post-process with external audio tools for more precise control

Generation is extremely slow

Cause: Running on CPU, or model is too large for available VRAM (causing swap).

Solutions:

1. Make sure the model is loaded on GPU, not CPU
2. Check that CUDA is properly installed: run `python -c "import torch; print(torch.cuda.is_available())"`
3. Unload other models to ensure full VRAM is available
4. For previews, use Kokoro (fastest) instead of Fish Speech (slowest)

Whisper verification keeps failing (retries endlessly)

Cause: Tolerance threshold too high, or the TTS output doesn't closely match the input text.

Solutions:

1. Lower the tolerance (try 0.72-0.78)
 2. For Kokoro: turn off Whisper verification or set it to 0.92+
 3. Simplify the input text -- remove complex punctuation, abbreviations
 4. Edit the `job.json` file and set `"verify_whisper": false` for the problematic chunk
 5. If all else fails, disable Whisper verification entirely
-

Job Recovery

How to recover a failed TTS job

1. Make sure the same model is loaded
2. Set the same save path as the original job
3. Type `##recover##` in the text area
4. Click Generate

The app reads `job.json` from the project directory and retries only failed chunks.

`job.json` doesn't exist

The job tracking file is only created when you use a save path. If you generated without a save path (temp output), there's no recovery file.

Some chunks keep failing after recovery

1. Open `job.json` in a text editor
 2. Find the failing chunk entries
 3. Try: lower the temperature, disable Whisper verification, or simplify the text for that chunk
 4. Save the file and run recovery again
-

Video Production Issues

"Create Video" button is disabled

Cause: No audio file is selected, or the audio file doesn't have an SRT subtitle file.

Solutions:

1. Select an audio file from the project files list
2. Transcribe it first (click "Transcribe with Whisper")
3. Verify that a `.srt` file exists alongside the audio file

Video creation fails with FFmpeg error

Cause: FFmpeg binary not found or misconfigured.

Solutions:

1. Verify `bin/ffmpeg/bin/` contains `ffmpeg.exe` and `ffprobe.exe`
2. Check the `FFMPEG_BIN` path in `config.py`
3. Check the terminal for the specific FFmpeg error message
4. Try a different resolution or background mode

Subtitles don't appear in the video

Cause: SRT file is empty or malformed.

Solutions:

1. Check the transcription text in the Production Studio -- is it empty?
2. Re-transcribe the audio file

3. Open the .srt file in a text editor to verify it has valid content
-

Chatbot Issues

Local Llama model won't load

Cause: Model file is corrupted, too large for memory, or wrong format.

Solutions:

1. Verify the file is a valid .gguf format
2. Try a smaller model
3. Reduce GPU layers to offload some work to CPU
4. Check LLM_DIRECTORY in config.py points to the correct folder

LM Studio connection fails

Cause: LM Studio isn't running, or the API address is wrong.

Solutions:

1. Make sure LM Studio is running and has a model loaded
2. Check that LMSTUDIO_API_BASE in config.py matches LM Studio's settings (default: <http://127.0.0.1:1234/v1>)
3. Try loading a model in LM Studio and refreshing the status

OpenRouter returns an error

Cause: Invalid API key, insufficient credits, or selected model unavailable.

Solutions:

1. Verify your API key in config.py
2. Check your OpenRouter account balance
3. Try a different model from the dropdown
4. Check OpenRouter's status page for outages

Chatbot response is cut off

Cause: Max tokens setting is too low.

Solutions:

1. Open the gear settings panel
 2. Increase the Max Tokens slider
 3. For local models, also ensure the Context Length is sufficient
-

Stable Audio / ACE-Step Issues

Generated audio is noisy or low quality

Solutions:

1. Increase the Steps parameter (try 80-100 for Stable Audio, 50-60 for ACE-Step)
2. Adjust the Guidance Scale (try 4-6)

3. Write a more specific prompt
4. Generate more variants and use CLAP scoring to pick the best

Output doesn't match the prompt

Solutions:

1. Increase the Guidance Scale for stronger prompt adherence
 2. Be more specific in your prompt description
 3. Use negative prompts to exclude unwanted elements
 4. Try different seeds
-

General Issues

App won't start

Common causes and solutions:

1. Port in use: Another process is using port 5006. Use --port 5007 or kill the other process.
2. Missing dependencies: Run `pip install -r requirements.txt` again
3. Python version: Ensure Python 3.9+
4. CUDA mismatch: PyTorch CUDA version must match your NVIDIA driver

Browser shows a blank page

1. Clear browser cache and refresh
2. Check the terminal for JavaScript errors
3. Try a different browser (Chrome, Firefox, Edge)

Files aren't being saved

1. Check that the save path is correct
2. Verify `projects_output/` directory exists and is writable
3. Check available disk space

App crashes during generation

1. Check the terminal for error messages
 2. Most likely cause is GPU out of memory -- unload unused models
 3. If the crash is reproducible, try the same text with different settings
 4. For TTS, use `###recover###` to resume
-

Chapter 16: Frequently Asked Questions

General

Q: Can I use LocalSoundsAPI without a GPU?

A: Yes. Every model supports CPU mode. Select "CPU" from the device dropdown before loading. Expect significantly slower generation times (roughly 10-50x slower than GPU for most models). Kokoro is the exception -- it runs at 30-40x real-time even on CPU, making it a great choice for GPU-free setups.

Q: Is my data private?

A: Yes, when using local models or LM Studio. All processing happens on your machine and nothing leaves it. The only exception is the OpenRouter chatbot backend, which sends your conversation to external cloud servers. The TTS engines, audio generators, and transcription always run locally.

Q: Can I host this publicly / on a server?

A: The app is designed for local use on 127.0.0.1. While it listens on 0.0.0.0 (all interfaces), it lacks authentication, rate limiting, and other security features needed for public deployment. If you want to use it on a local network, understand that anyone on your network can access it. Do not expose it to the open internet without adding proper security.

Q: How do I update the models to newer versions?

A: Delete the model folder (e.g., models/XTTS-v2/) and restart the app. The next time you click "Load," it will download the latest version from Hugging Face.

Q: Can I run multiple instances at the same time?

A: Yes. Start each instance on a different port:

```
python main.py --port 5006
python main.py --port 5007
```

Each instance is fully independent. Be careful about GPU memory -- two instances loading the same model will use double the VRAM.

Text-to-Speech

Q: What's the difference between XTTS, Fish Speech, and Kokoro?

A:

	XTTS-v2	Fish Speech	Kokoro
Speed	Medium	Slow	Very fast
Quality	Very good	Excellent	Good
Voice cloning	Yes	Yes (best quality)	No
Preset voices	59	None	19
VRAM	~6 GB	~8 GB	~2 GB
CPU usable	Yes (slow)	Yes (very slow)	Yes (fast!)
Best for	General use	Final production	Quick previews

Quick guide:

- Need voice cloning? Start with XTTS, use Fish Speech for final output
- Need speed? Use Kokoro
- Need quality? Use Fish Speech
- Need flexibility? Use XTTS

Q: How long should my voice reference file be?

A: 10-30 seconds for XTTS, 8-20 seconds for Fish Speech. The reference should be:

- Clean speech without background noise or music
- Natural conversational tone (not monotone reading)
- Single speaker only
- Clear recording quality

Q: Why does generation keep retrying the same chunk?

A: Whisper verification is rejecting the output. Try:

1. Lowering the tolerance threshold (e.g., from 0.80 to 0.74)
2. Lowering the temperature
3. Simplifying the text for that section
4. Disabling Whisper verification if speed matters more than perfect accuracy

Q: What does `##recover##` do?

A: When you type `##recover##` in the text area and click Generate, the app looks for a `job.json` file in your project directory. This file tracks the status of each text chunk. The recovery process retries only the chunks that failed, keeping all successful chunks intact. It's like resuming a download instead of starting over.

Q: Can I generate speech in languages other than English?

A: XTTS-v2 supports multiple languages. Fish Speech also has multilingual support. Kokoro is English-only (it uses eSpeak-ng for phoneme conversion). For non-English text, XTTS is your best bet.

Audio Generation

Q: How do I get better sound effects from Stable Audio?

A:

1. Be specific in your prompts (describe instruments, texture, tempo, mood)

2. Use negative prompts to exclude unwanted elements
3. Generate 3-4 variants and let CLAP pick the best one
4. Keep SFX short (5-12 seconds)
5. Use 80-100 steps for good quality
6. Experiment with the audio mode (Impact vs Ambient vs Music)

Q: Can ACE-Step generate full songs with vocals?

A: ACE-Step can generate music with sung or rapped lyrics. Write the first line as a style description and the remaining lines as lyrics. The quality is acceptable but not studio-grade. For instrumental music, omit the lyrics or include "instrumental" in the style line.

Q: What is the CLAP score?

A: CLAP (Contrastive Language-Audio Pretraining) is a model that scores how well an audio clip matches a text description. When you generate multiple variants, CLAP scores each one and ranks them. Higher scores mean the audio better matches your prompt. It's not perfect, but it's a useful automatic quality filter.

Video Production

Q: What video formats are supported?

A: Output is MP4 (for solid color and image backgrounds) or WebM (for transparent background). Input media can be any format FFmpeg supports, including MP4, WebM, MOV, AVI, MKV for video and JPG, PNG, GIF, BMP, WebP for images.

Q: Can I edit the subtitles before creating the video?

A: The SRT file is a standard text file that you can edit with any text editor. After editing, create the video as normal -- it will use the modified SRT file. The transcription display in the app is read-only, but the underlying file is fully editable.

Q: How does the "Cycle Images" background mode work?

A: It takes all image and video files in your project folder and uses them as rotating backgrounds behind the subtitles. Images are displayed in sequence, each for an equal portion of the audio duration. This creates a slideshow-style video automatically.

Chatbot

Q: What .gguf models work with the local Llama backend?

A: Any GGUF format model compatible with llama-cpp-python. Popular options include Llama, Mistral, Phi, and other open-source models in GGUF format. The GPU Layers slider lets you control how much of the model is offloaded to your GPU.

Q: Can I use the chatbot while generating TTS?

A: Yes, if they're on different devices or using different backends. For example, run TTS on GPU 0 and the chatbot via LM Studio or OpenRouter. Running both on the same GPU may cause memory issues.

Q: Where are my conversations stored?

A:

- Current conversation: brain/context_history/current.json
- Archived conversations: brain/context_history/archives/
- System prompts: brain/system_prompt.json and brain/[preset_name].json

All files are standard JSON and can be backed up, edited, or transferred.

Performance

Q: How much disk space do I need?

A:

- Application code: ~50 MB
- All models (if downloaded): ~30-40 GB
- Projects and output: varies by usage
- Total recommended: 100-200 GB free

Q: How can I make generation faster?

A:

1. Use GPU instead of CPU
2. Use Kokoro for quick previews, Fish Speech for final output
3. Unload unused models to free VRAM
4. Reduce inference steps (for Stable Audio / ACE-Step)
5. Disable Whisper verification when speed matters more than perfect accuracy
6. Use a smaller Whisper model (base.en instead of medium.en)

Q: Can I use AMD GPUs?

A: PyTorch has experimental ROCm support for AMD GPUs on Linux. On Windows, AMD GPU support is limited. Check the PyTorch documentation for the latest AMD compatibility.

Chapter 17: Launcher (GUI Instance Manager)

The LocalSoundsAPI Launcher is a tkinter desktop application that replaces the batch-file workflow with a single window for managing multiple server instances, downloading AI models, and monitoring logs.

Starting the Launcher

Double-click launcher.bat or run it from a terminal:

```
launcher.bat
```

This sets up the PATH for the portable Python environment, FFmpeg, and RubberBand, then opens the launcher window.

Tab 1: Instances & Log

This is the main tab where you manage server instances and view their output.

Adding an Instance

At the top of the tab you'll find:

- 1. Device dropdown -- Lists all detected NVIDIA GPUs plus a "CPU" option. GPUs are auto-detected using nvidia-smi.
- 2. Port field -- The port number for this instance. Defaults to 5006 and auto-increments after each add.
- 3. Add Instance button -- Creates the instance entry in the table below.

Each instance runs an independent copy of LocalSoundsAPI on the selected GPU and port.

Instance Table

The table shows all configured instances with columns:

Column	Description
Device	The GPU or CPU assigned to this instance
Port	The network port (e.g., 5006)
Status	Stopped, Starting, Running, or Crashed
URL	The address to access this instance in a browser

Instance Controls

Buttons below the table:

Button	Action
Start	Launch the selected instance as a background process
Stop	Terminate the selected instance
Start All	Launch all stopped instances
Stop All	Terminate all running instances

Remove	Delete the selected instance from the table (stops it first if running)
Open Browser	Open the instance URL in your default web browser

You can also double-click any row in the table to open it in your browser.

How Instances Run

Unlike the batch files (which open separate cmd windows), the launcher runs each instance as a hidden subprocess. All output is captured and displayed in the log viewer below. The subprocess is started with:

- CUDA_VISIBLE_DEVICES set to the selected GPU index (for GPU isolation)
- CREATE_NO_WINDOW flag (no separate console window)
- Unbuffered output (-u flag) for real-time log streaming

Log Viewer

The dark-themed log panel at the bottom shows timestamped output from all instances. Features:

- Color-coded entries -- System messages (yellow/gold) vs server output (light gray)
- Timestamps -- Every line prefixed with [HH:MM:SS]
- Instance prefixes -- Each line tagged with [GPU0:5006] (or similar) so you can tell which instance produced it
- Filter dropdown -- Show logs from all instances or filter to a specific one
- Clear button -- Wipe the log display
- Line count -- Shows total visible lines
- Auto-scroll -- Automatically follows new output
- Maximum 5000 lines -- Older entries are trimmed to prevent memory issues

Health Polling

Every 15 seconds, the launcher checks each running instance by calling its /status endpoint. If an instance responds, the status shows "Running". If it doesn't respond but the process is alive, it shows "Starting". If the process has exited unexpectedly, the status changes to "Crashed".

Tab 2: Models & Tools

This tab provides a dashboard for managing the portable environment, tools, and AI models.

Portable Python Environment

Shows the current Python version and path. The Download / Reinstall button downloads portable-python-env-v1.7z from the GitHub releases page and extracts it to the project root. This is a repair/reinstall feature -- the launcher itself is already running on the portable Python.

Portable Tools (bin/)

Displays status indicators for the three required external tools:

Tool	What it checks
FFmpeg	bin/ffmpeg/bin/ffmpeg.exe exists
RubberBand	bin/rubberband/ directory exists
eSpeak-ng	bin/espeak-ng/libespeak-ng.dll exists

Each tool shows a green dot if found or a red dot if missing.

The Download All Tools button downloads bin.zip from the GitHub releases page and extracts it into the project root, installing all three tools at once.

AI Models

A table showing all downloadable AI models:

Model	Check Path	Approximate Size
XTTS-v2	models/XTTS-v2/	~6 GB
Fish Speech	models/fish-speech/	~8 GB
Kokoro-82M	models/kokoro-82m/	~500 MB
Whisper (medium.en)	models/medium.en.pt	~1.5 GB
Stable Audio	models/stable-audio-open-1.0/	~10 GB
ACE-Step	models/ace_step/	~7 GB
CLAP	models/clap-htsat-unfused/	~600 MB

Each model shows its status:

- Installed -- The model files exist on disk
- Not Found -- The model hasn't been downloaded yet

Select a model and click Download Selected to begin downloading. Models are downloaded from Hugging Face using `huggingface_hub.snapshot_download()` (or the OpenAI Whisper library for Whisper). A progress bar at the bottom of the tab shows download activity.

Download Progress

A progress bar and status label at the bottom of the Models & Tools tab shows:

- Download percentage for tools/Python env (determinate progress)
- Indeterminate animation for model downloads (since `huggingface_hub` doesn't report granular progress)
- Extraction status after download completes

Only one download can run at a time.

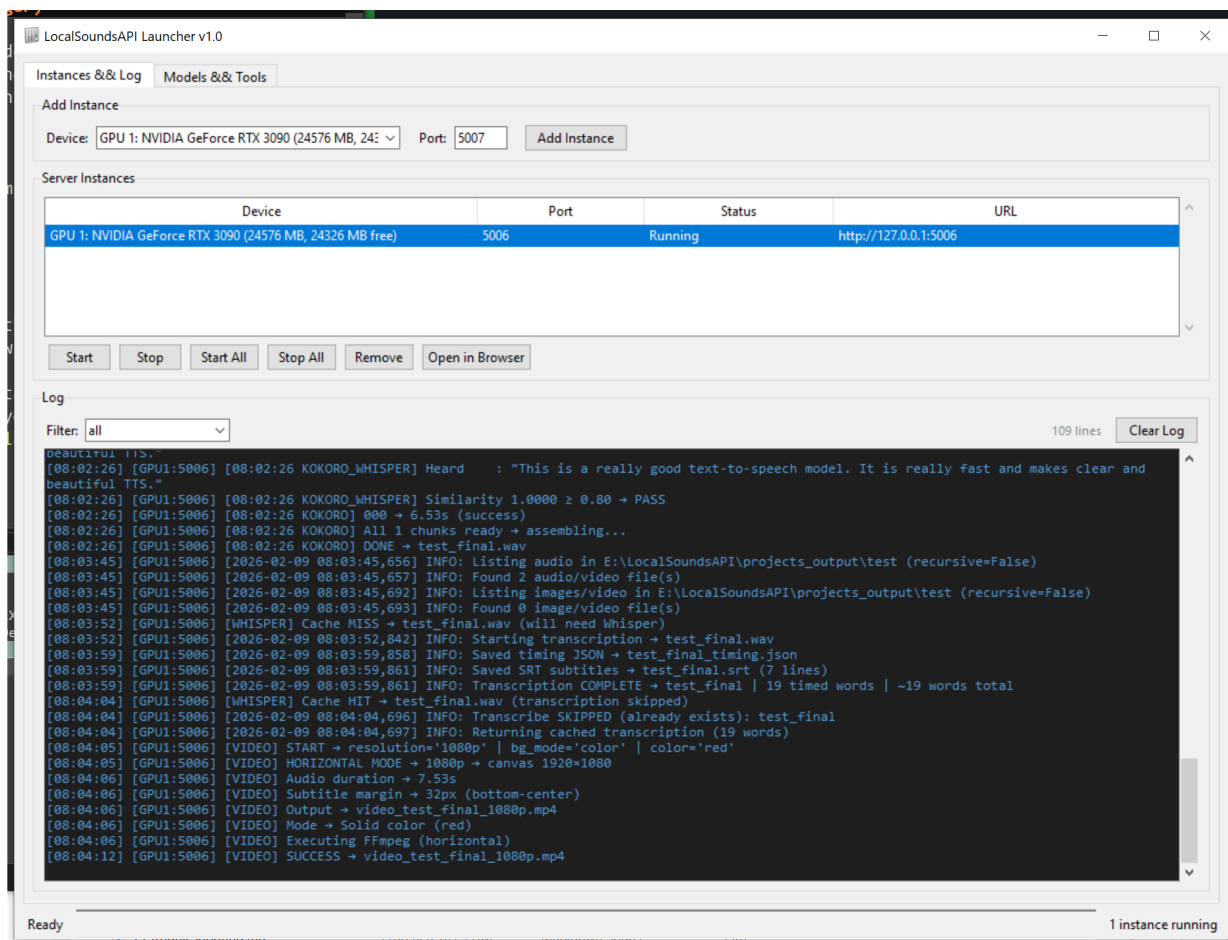
Closing the Launcher

When you close the launcher window:

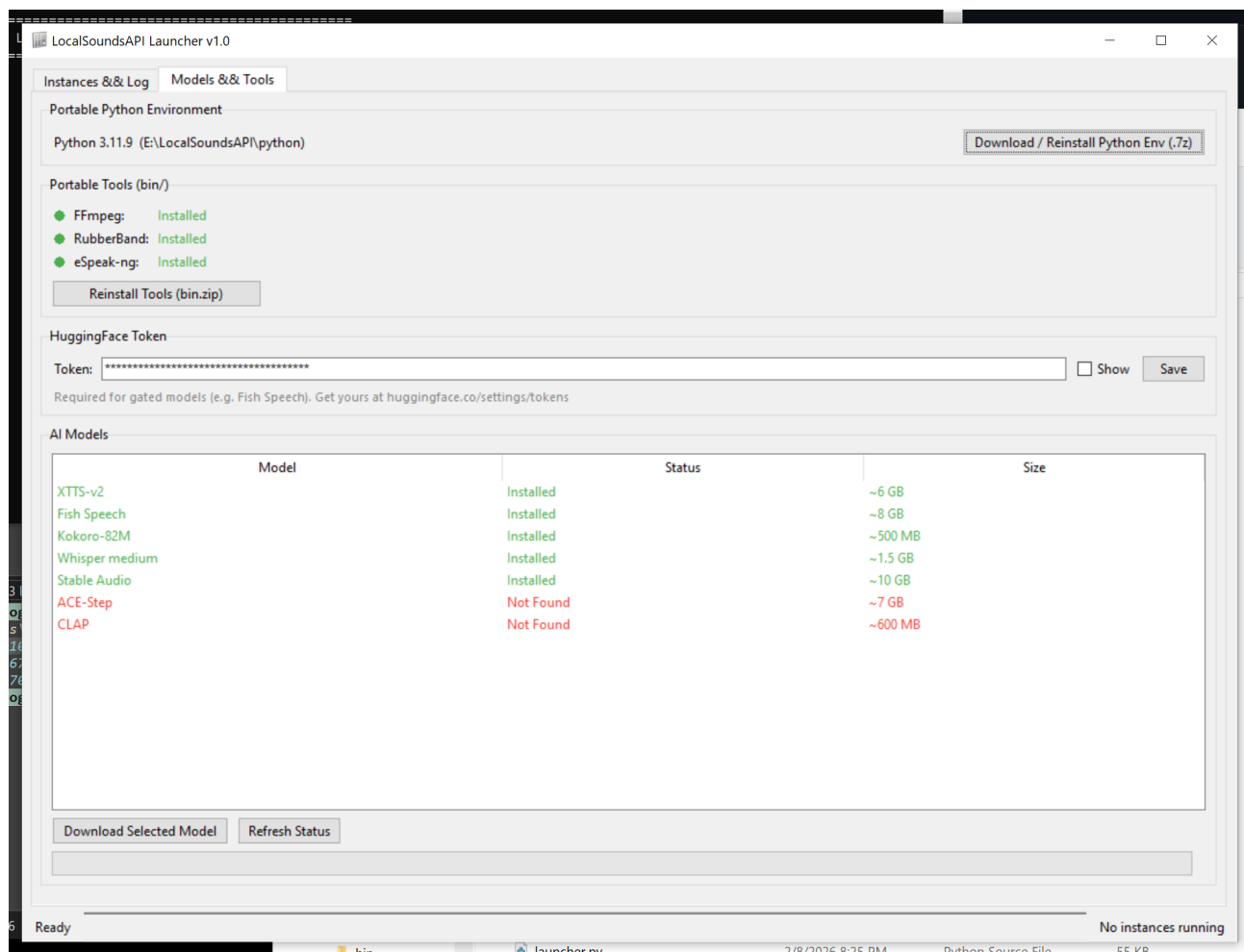
1. If any instances are running, a confirmation dialog appears
2. All running instances are stopped gracefully (using `taskkill` on Windows)
3. The window closes

Tips

- Port collisions -- The launcher prevents you from adding two instances on the same port.
- GPU memory -- Loading the same model on two instances using the same GPU will use double the VRAM. Use different GPUs for different instances when possible.
- CPU fallback -- If no NVIDIA GPU is detected, only "CPU" appears in the device dropdown.
- Batch files still work -- The existing .bat files continue to work alongside the launcher. Use whichever approach you prefer.
- Crashed instances -- If an instance status shows "Crashed", check the log for error messages. Common causes are port conflicts and out-of-memory errors.



The Instances & Log tab with a running instance on GPU 1 (RTX 3090) at port 5006. The log viewer shows Flask startup output, model loading, TTS generation, Whisper verification, and video creation -- all captured with timestamped [GPU1:5006] prefixes.



The Models & Tools tab showing the portable Python environment, tool status indicators (all green), HuggingFace token entry, and the AI model table with Installed/Not Found status for each model.